UMI

A Trust-Minimized ASL-to-English Translation Subnet

Public whitepaper and conformance specification

Protocol version	0.1
Network	Bittensor mainnet, subnet 78
Status	SN78 active on mainnet; UMI translation weights inactive; calibration profile published; public calibration pending
Publication date	August 2026

This edition supersedes earlier UMI mechanism and whitepaper drafts.

Contents

Abstract

This whitepaper defines UMI, a Bittensor subnet that measures and rewards systems for translating raw American Sign Language video into English text. Miners perform the translation. Validators independently challenge and score them against references that were committed before assignment and concealed through a dedicated reveal after responses close. Miner answers remain sealed until that same reveal. Deterministic text metrics, single-use challenges, signed records, and reproducible audit bundles minimize the trust placed in any publisher or validator.

ASL-to-English is UMI's first protocol scope. UMI's broader objective is verifiable, bidirectional translation between people and machines: interpreting human language, motion, gesture, expression, and context as machine-usable representations, and rendering machine state or intent in forms people can understand and act on.

When activated, UMI translation weights use one emission-bearing mechanism by design. This gives the network one reproducible ranking after the task, data supply, and adversarial controls pass calibration. The chain supports multiple mechanisms. Any later expansion is a governed protocol extension with its own evidence, threat analysis, emission policy, and UID budget.

This is a pre-weight-activation specification, not a claim that the launch gates have already passed. At publication, publisher independence, positive miner utility at the initial floor, and validator operating sustainability remain to be demonstrated publicly.

1 Mechanism overview

The launch protocol evaluates one useful output: English text translated from raw ASL video. It supports end-to-end video models, pose-based systems, hosted APIs, and model ensembles without prescribing how a miner produces its answer.

A scoring cycle has six stages:

1. A challenge publisher prepares consented ASL clips and independent English references, encrypts the references to a future reveal round, and anchors one canonical pool manifest for the selection window.
2. After the pool of eligible batches closes, validators use a post-close randomness round to select batches and miners. This prevents publishers or validators from choosing work after seeing the selection seed.
3. Validators anchor their exact assignment set, then send authenticated raw-video challenges. Miners return bounded, signed answer envelopes timelocked to the ground-truth `reveal_round`.
4. After `response_close_round`, validators anchor the exact sealed envelope or failure marker for every request. At `reveal_round`, they open miner answers and committed references together, then score every assignment with the same deterministic character error rate (CER) or word error rate (WER) policy. Missing or invalid work remains in the denominator and scores zero.
5. Each validator computes its scoring and weight materials. Under the shadow policy it records the projected row without submitting it. Under the activated policy it submits through the chain-native timelocked path.
6. An activated validator publishes its signed evidence after chain auto-reveal and verification of the exact applied MechId 0 row. A shadow validator publishes after the corresponding commit window closes. A terminally failed reveal produces an incident bundle instead. Yuma Consensus combines applied validator weights into incentives and dividends.

Scoring applies only to translation services. End-user application behavior is outside scope. The protocol assumes no trusted scoring API or trusted hardware environment. Human review remains necessary to establish reference quality, while the emission-bearing calculation after reveal is deterministic and independently reproducible.

The key words MUST, MUST NOT, SHOULD, SHOULD NOT, and MAY describe conformance requirements throughout this whitepaper.

1.1 Terminology

- **Script**: the English prompt a contributor is asked to convey in ASL. The normalized script hash keys freshness and retirement.
- **Script group**: every clip recorded from one script, together with that script's reference sets, across all signers.

- **Reference**: an accepted English rendering of a clip, fixed before batch commitment. References may differ from the script; the script prompts the signing, and the references describe what the clip actually conveys.
- **Batch**: a sealed set of clips and references committed as one unit by one publisher.
- **Reveal cohort**: every batch whose ground truth opens at the same randomness round.
- **Stratum**: a linguistic difficulty class (Section 7.2). The stratum label is intentionally visible in the challenge request; the task type is not a secret, the content is.
- **Selection window**: one candidate-pool close, future randomness round, and scoring cycle under one policy hash.
- **Pool manifest**: a publisher's ordered list of candidate batches for one selection window, committed by hash before the window closes.
- **Availability certificate**: a quorum of validator signatures confirming that the signed pool artifacts were retrieved, verified, and mirrored before close.
- **Publisher control group**: publisher hotkeys under common ownership, administration, funding, or ground-truth access. Exposure caps apply to the group, not merely to its hotkeys.
- **Spent registry**: the deterministic append-only state derived from finalized pool anchors and revealed batch artifacts. It is calculated, not edited by a publisher or validator.
- **Publisher fault registry**: deterministic state derived from objectively invalid revealed publisher artifacts. It controls cooldown and version-level exclusion without a validator vote.

2 Launch profile and design principles

Topic	Version 0.1 decision
Language pair	ASL (`ase`) to English (`en`)
Primary task	Raw video to English text
Long-term objective	Verifiable bidirectional translation across human and machine interaction modes
On-chain mechanisms	One by design, MechId 0
Later mechanisms	Governed extension only; no launch emission reserved
Emission-bearing metric	Deterministic CER or WER against a sealed reference set
Challenge source	Consented, quality-reviewed, fresh human recordings
Challenge reuse	A revealed script group and its clips are permanently spent
Leak detection	Sealed canary items plus source-conditioned shadow monitoring
Pose extraction	Input-quality and research diagnostic, zero score weight
Semantic similarity	Shadow metric, zero score weight
LLM judging	Excluded from weight calculation
Miner confidence	Excluded from weight calculation
Latency	Measured in shadow mode until network effects are calibrated
Corpus contribution	External data pipeline, outside subnet weights in version 0.1

One mechanism keeps launch incentives aligned with the useful output and gives validators one ranking to reproduce. The chain permits multiple mechanisms; version 0.1 deliberately uses one. Later mechanisms require a protocol amendment and evidence that their task produces independent value.

2.1 Launch status

Version 0.1 separates a specified control from evidence that the control is ready:

Question	Status at publication
Who holds the answer key?	Before independent groups are active, the founding publisher control group may prepare every calibration reference. All hotkeys under that common administration count as one control group. Locks, canaries, and audit bundles make behavior more attributable; they do not make that source independent.
Do models clear the `0.10` floor?	Not yet demonstrated under UMI's exact normalization, reference, stratum, assignment-failure, and rolling-score rules. The value is an initial calibration candidate, not a reported model result.

Question	Status at publication
Who bears single-use challenge cost?	Each publisher bears the production and review cost of its candidate batches and receives no subnet emission for publishing them. Publishers in the founding control group therefore bear that group's cost. Only benchmark eligibility is spent after reveal; the underlying data is not destroyed, although any later use still depends on its consent class.
Does validation guarantee profit?	No. A permitted conforming validator is eligible for variable Yuma dividends, but realized operator return also depends on stake, bonds, consensus, delegation, subnet emission, alpha liquidity and price, and direct operating cost. A calibration run, even on mainnet, does not guarantee future dividend revenue.

UMI translation weights MUST remain inactive until Section 14's independent publisher, miner-utility, challenge-supply, and validator-economics gates pass. If a gate does not pass, the protocol pauses rather than replacing missing evidence with a centralized assertion or a fallback weight row.

2.2 Trust-minimized design rules

Version 0.1 follows these rules:

1. The emission-bearing score is a pure function of committed inputs, signed responses, revealed references, and a versioned scoring policy.
2. Every conforming validator computes its own scores from source evidence. Shared score and ranking APIs are ineligible as validator inputs.
3. Ground truth is fixed before miner assignment and concealed until all responses close.
4. The candidate pool closes before the selection seed exists.
5. Missing work remains in the denominator and scores zero.
6. Freshness, uniqueness, and reliability are eligibility conditions with fail-closed outcomes.
7. Activated weight submission uses chain-native timelocked commit-reveal.
8. Public audit evidence is sufficient to reproduce pre-quantization weights exactly.
9. Owner-controlled parameters cannot regrade a committed batch.
10. A trusted hardware environment is absent from the scoring trust boundary.
11. Score and weight evidence becomes public only after the validator's timelocked weights are revealed and applied on chain, or after its exact pending entry has been removed without application. A validator that never submits waits through the shared commit close before publishing outcome evidence.
12. Publisher eligibility requires chain-locked collateral. Source-conditioned score anomalies remain public shadow telemetry in version 0.1 and cannot remove assigned work from the denominator.
13. Candidate-pool membership and spent state derive from finalized chain state under one versioned policy; local availability never changes either set.
14. Miner answers remain timelock-sealed until the ground-truth reveal. Validators anchor the sealed response set first, so they cannot read and insert copied answers after responses close.

2.3 Adopted subnet patterns

The launch mechanism combines patterns used by operational Bittensor systems without copying their task-specific assumptions.

Source pattern	Use in this protocol
Score Vision: structured video outputs and lightweight sampled validation	Bounded schemas and sparse motion diagnostics designed to keep validation cheaper than miner inference
Apex: identical sandboxed objective functions across validators	One canonical scoring package, fixtures, and exact score reproduction
OpenRoboto: hash-pinned artifacts and evaluation entropy fixed after submission	Batch commitments before selection, post-close entropy, and content-addressed audit evidence

Source pattern	Use in this protocol
Chutes audit: long-window reliability and inspectable work records	Rolling scores, assigned failures, signed records, and separate availability telemetry
Data Universe: freshness, uniqueness, and credibility	Single-use challenges, deduplication, source diversity, and publisher reliability gates

The protocol also follows Bittensor's current Yuma Consensus, signed-request, and validator weight interfaces.

3 Scope

Version 0.1 covers:

- short-form and continuous ASL video;
- English text output;
- fresh challenge creation and retirement;
- authenticated miner inference over HTTP;
- deterministic scoring and validator weight production;
- public score recomputation after ground-truth reveal;
- consent, provenance, retention, and audit requirements.

UMI's long-term scope is broader. The following remain outside version 0.1:

- English-to-sign generation and avatars;
- speech recognition and speech synthesis;
- sign languages other than ASL;
- high-consequence medical, legal, or financial use;
- claims of interpreter equivalence or accessibility certification;
- emission-bearing human opinion scores;
- contributor recruitment and administration beyond the eligibility rules below.

4 Actors and trust boundaries

4.1 Miner

A miner serves an authenticated translation endpoint and returns one English hypothesis for each assigned video. Miner internals are unrestricted. A miner MAY call an external model or combine several models.

4.2 Validator

A validator selects eligible batches and miners, anchors and issues challenges, verifies signed response envelopes, anchors them after response close, opens miner answers and ground truth at the reveal round, and calculates scores. Under an activated policy it submits a timelocked weight commitment and verifies the finalized auto-reveal and applied row. Under a shadow policy it stops after the projected weight build. In either case, it publishes the applicable audit bundle.

4.3 Challenge publisher

A challenge publisher creates eligible clips and reference sets, timelock-encrypts ground truth, publishes the encrypted artifacts, and anchors its pool-manifest hash before the selection window closes.

Publisher eligibility requires Bittensor's voluntary collateral. Ordinary liquid stake is ineligible. A publisher MUST use a dedicated registered hotkey that does not mine or validate in version 0.1. Its coldkey funds the actual lock with `add_collateral` and sets a self-maintaining floor with `set_min_collateral`. When collateral acquisition executes a pool buy, the publisher MUST use the official MEV-shielded submission path when it is live on that network and verify the finalized inner call.

At finalized closing block `B`, the hotkey MUST be registered, its current owner MUST equal the registry coldkey, and `MinerCollateral(netuid, hotkey, coldkey)` MUST report both `locked>=M_alpha` and `min_locked>=M_alpha`, where the active policy declares `M_alpha`. All reads are pinned to `B`. Missing state, unsupported runtime fields, owner mismatch, deregistration, or either failed inequality makes the pool anchor ineligible. The mutable floor alone is never evidence that collateral was funded.

The publisher hotkey earns no emission during compliant operation, so its lock does not drain. Clearing the floor creates no direct withdrawal path; without later emission, the collateral remains locked. This is a standing participation cost. Version 0.1 provides no escrow, slashing, or transfer to a harmed party. The active publisher registry is an ordered list of publisher hotkeys, owning coldkeys, and declared control-group IDs in the scoring policy. Hotkeys with common ownership, administration, funding, or ground-truth access MUST share one group. Adding an entry, removing an entry, or changing a group requires a new policy hash and activation block and cannot affect an open selection window. A new identity is a new governance decision backed by fresh collateral; the protocol does not infer off-chain succession or treat a new hotkey as independent of its controller. Control-group IDs are immutable within protocol version 0.1. A later policy MAY merge groups only through a canonical alias record that names every predecessor and inherits the union of their fault leaves, the sum of their strike counts saturated at two, and the latest cooldown end. It MUST NOT split or rename a struck group. Every validator derives the same alias transition from the signed policy bytes.

During pre-weight-activation calibration on SN78, one founding publisher control group MAY operate every available publisher hotkey. Because those hotkeys share administration and ground truth, they remain one group and cannot produce a conforming two-group scoring window. They support component tests and explicitly labeled simulations only. UMI translation-weight activation requires the independent control-group gate in Section 14. Until that gate is met, collateral, canaries, and public audit support testing but do not establish publisher independence.

Challenge production is an external protocol input and receives no protocol-assigned publisher emission in version 0.1. Each publisher bears the cost of every batch it proposes, including an unselected batch that retires unused. Before UMI weight activation, each independent group MUST sign a capacity statement covering the policy's `challenge_supply_runway_days` at the proposed activated cadence and the loss of any one other group. The statement uses this logical schema:

```
{
  "schema": "umi-publisher-capacity/1",
  "control_group_id": "hex-encoded-32-bytes",
  "administrator": "ss58-policy-declared-group-administrator",
  "publisher_hotkeys": ["ss58-sorted-publisher-hotkey"],
  "scoring_policy_hash": "hex-encoded-sha256",
  "activation_equivalence_digest": "hex-encoded-sha256",
  "issued_block": 123456,
  "issued_block_hash": "0x...",
  "valid_from_block": 123456,
  "valid_through_block": 771456,
  "cadence": {
    "window_stride_blocks": 360,
    "target_block_interval_seconds": 12,
    "scheduled_windows": 1800
  },
  "per_window_capacity": {
    "candidate_batches": 1,
    "emission_bearing_clips": 12,
    "canary_clips": 2,
    "delivered_clips": 14,
    "maximum_retired_script_groups": 16
  },
  "runway_totals": {
    "candidate_batches": 1800,
    "delivered_clips": 25200,
    "maximum_retired_script_groups": 28800
  },
  "one_group_loss": {
    "minimum_remaining_groups": 2,
    "this_group_continues_at_declared_capacity": true
  },
  "control_disclosure_sha256": "hex-encoded-sha256"
}
```

The numbers above illustrate the initial 360-block cadence; the signed object MUST contain the exact proposed activated cadence and arithmetically derived totals. The active policy declares one administrator account per control group. Publisher hotkeys are sorted by decoded account, and the administrator signs:

```
publisher_capacity_digest = SHA256(
  "umi-publisher-capacity-v1\0" || RFC8785(capacity_statement)
)
```

The signature follows Section 6.3. Its validity interval MUST cover the full runway from the proposed activation block. The statement reports aggregate batch, clip, canary, and reserved-script capacity, not contributor or reviewer identities. Future human availability and funding remain off-chain assertions; the protocol can authenticate the commitment and expose realized supply and retirement but cannot cryptographically prove future runway. An emission-bearing corpus mechanism would require the separate activation process in Section 16.

4.4 Contributor and reviewer

A contributor records a prompted ASL clip under an explicit consent policy. Independent reviewers see the clip without its prompt and provide English reconstructions or reject the sample. These roles operate through the data pipeline in version 0.1.

4.5 Auditor

An auditor verifies batch ordering, request and response anchors, ground-truth reveal, score calculation, challenge retirement, UID resolution, and either the projected shadow row or the applied activated row.

4.6 Participation economics

At publication, Bittensor assigns roughly 18% of participant-side subnet alpha to the current subnet owner when `owner_cut_enabled` is true, then divides the remainder equally between miner incentive and the validator-side allocation. A variable root-staker share can reduce what subnet validators and their stakers receive. The live emission rules, owner-cut state, Yuma mode, stake, bonds, validator take, and pool state are authoritative at UMI weight activation. Owner-side revenue is not automatically earmarked for challenge data. Subject to realized receipt and liquidity, the owner MAY use it to support the founding publisher control group; that does not create a protocol entitlement for any publisher.

A conforming validator with a live permit can earn Yuma dividends, but conformance alone does not establish positive operating return. The 30-day soak MUST therefore produce one public `umi-validator-economics/1` report for every validator in the policy's registered soak set, which contains at least four independently administered validators. The report binds the validator hotkey, shadow policy hash, activation-equivalence digest, resource-capacity statement hash, resource measurements, cost-schedule hash, finalized chain and pool snapshots, dividend replay, break-even result, and every conversion input.

Before the soak, the shadow policy MUST pin a versioned common cost schedule. It declares one reporting currency, hardware and region classes, unit definitions, the exact conversion functions, and the deterministic observation rule for each price source. Each non-chain unit cost is the greatest of at least three independently published list prices for the same class, captured with source, timestamp, and content hash. At every required snapshot, `P_b` is the least reporting-currency price per TAO from at least three independently published spot sources under that record format. A missing source or observation fails the gate. Actual finalized chain fees replace list prices for chain calls.

Each report contains measured request count, transferred and retained bytes, compute time by declared hardware class, chain calls and fees, the validator's live take, and a reproducible conservative dividend case. That case replays the soak's weight rows through the policy-pinned Yuma implementation using the activation-block per-coldkey stake and delegation, parent-child routing, takes, participant-side alpha emission, root share, mode, and exact MechId 0 bond state, all backed by storage proofs.

For projected 30-day operator dividend `x` alpha, let `Q_b(x)` be the canonical executable TAO output at finalized pool snapshot `b`, including the live fee and price impact. The required soak snapshot for every scheduled window is its finalized `closing_block`; the activation snapshot is the finalized activation block. Each report uses exactly those blocks, once each, with no discretionary or additional sample. Sort the complete soak values of `Q_b(x)` and

`P_b` independently in ascending order and take the nearest-rank `validator_quote_percentile`, with rank `max(1, ceil(p*n))`. The conservative alpha value in the reporting currency is:

```
validator_alpha_value(x) = validator_alpha_value_haircut * min(
  Q_activation_block(x),
  percentile_soak_Q(x)
) * min(
  P_activation_block,
  percentile_soak_P
)
```

The report states the direct-cost break-even dividend in alpha and every conversion separately. Native alpha distributed during calibration is reported as bootstrap emission, not as evidence of sustainable UMI revenue.

The gate passes only if every validator in the registered soak set has a conservative projected operator share covering at least `validator_cost_coverage` times its measured direct operating cost. Capital opportunity cost, token-price movement, pool liquidity, and future stake competition remain risks rather than guarantees. For every rolling 30-day period after UMI weight activation, every registered validator MUST publish the same measured quantities and realized gross alpha dividend. Falling below the coverage threshold does not alter scores, but it triggers the incident and policy-review procedure if fewer than four independently administered conforming validators remain economically covered.

4.7 Residual trust

Version 0.1 makes these trust assumptions explicit:

- A challenge publisher knows the ground truth and could leak it.
- Human review establishes reference quality and cannot be reduced to a text metric.
- Video delivery and audit artifacts use off-chain storage.
- Bittensor consensus assumes enough independent validator stake follows the protocol.
- Availability-certificate uniqueness assumes fewer than one third of the active validator registry equivocates.
- Publisher control groups depend on disclosed off-chain relationships; the chain cannot detect undisclosed common control.
- Validators cannot observe every failed availability attempt. A publisher can conceal that it disclosed an uncertified batch and later reuse the material; the publisher-local quarantine below makes reuse non-conforming but cannot make the disclosure history complete or trustless.
- Timelock availability depends on the pinned drand Quicknet and artifact mirrors.
- The current owned finality client verifies GRANDPA internally but emits a hash-pinned verifier attestation rather than portable GRANDPA proof bytes. It explicitly trusts the governed, policy-pinned Finney `grandpaWarpSyncCheckpoint` and its authority set as a weak-subjectivity input, then verifies peer-supplied warp and GRANDPA data from that point forward.
- Response timelocks prevent a validator from reading an answer early; they cannot prevent a miner from voluntarily sharing its own plaintext with another miner.
- Consent attestations establish authorization; the chain cannot prove that a person gave informed consent.
- Publisher capacity statements and validator resource meters contain off-chain assertions even when their counts and byte totals can be checked against public bundles.

Locked publisher collateral, canary items, control-group caps, single-use challenges, signed manifests, and post-reveal audits bound these risks. They do not remove them. Publisher collusion with a miner remains the largest residual attack surface in version 0.1. Section 9.8 makes source-conditioned anomalies public, but an anomaly alone neither proves collusion nor changes a committed score.

5 Version identifiers

Every scored request MUST bind the following versions:

Identifier	Launch value or source
Application protocol	`umi-asl/0.1`

Identifier	Launch value or source
Ground-truth schema	`umi-ground-truth/1`
Pool manifest schema	`umi-pool-manifest/1`
Mirror discovery schema	`umi-mirror-discovery/1`
Video-delivery issuance schemas	`umi-video-delivery-issuance-request/1`, `umi-video-delivery-issuance-response/1`, and `umi-video-delivery-issuance-evidence/1`
Availability certificate	`umi-availability/1`
Publisher capacity schema	`umi-publisher-capacity/1`
Validator economics schema	`umi-validator-economics/1`
Validator resource-capacity schema	`umi-validator-capacity/1`
Spent-registry schema	`umi-spent-registry/1`
Publisher-fault schema	`umi-publisher-fault/1`
Activation-drill schema	`umi-activation-drill/1`
Assignment-set schema	`umi-assignment-set/1`
Request-set schema	`umi-request-set/1`
Response-set schema	`umi-response-set/1`
Response envelope schema	`umi-response-envelope/1`
Response plaintext schema	`umi-response-plaintext/1`
Response timelock	`umi-response-tle/1`: `umi-tle/1` envelope to the ground-truth `reveal_round`
Batch timelock	`umi-tle/1`: Bittensor portable envelope over the policy-pinned drand quicknet
Media decoder	Content hash of the conformance decoder and frame-digest fixtures
Scoring policy	SHA-256 of the canonical policy document
HTTP authentication	`btauth/1`
Chain runtime	Runtime metadata and spec version at a pinned block
Reference implementation baseline	Subtensor runtime spec 449 at `71136ad`; activation state remains authoritative
Subnet weights	Current `WeightsVersionKey`
Weight concealment	Current chain commit-reveal version

A breaking request, pool, mirror, video-delivery, ground-truth, timelock, registry, or scoring change MUST increment its version. A scoring change MUST declare an activation block and MUST NOT alter any selection window opened under an earlier policy hash.

6 Miner API

6.1 Transport

Miners MUST expose `POST/v1/translate` over HTTPS at their announced serving endpoint. The certificate MUST be valid for the exact IP address in the finalized serving record under the validator's system trust store. Requests MUST use `btauth/1`. Deployments with several server processes MUST share replay state.

Validators MUST sign the exact request bytes. Miners MUST bind each response to the serving hotkey with a signature over the canonical response digest. JSON canonicalization follows RFC 8785.

Each request MUST carry exactly one `X-UMI-Body-SHA256` header containing the lowercase hexadecimal SHA-256 of the exact body. This is the body-hash line already covered by the `btauth/1` signature, not a second authentication scheme. Before reading the body, the miner verifies the canonical authentication-header set, receiver, freshness, signature, and policy validator account against that declared digest. It then reads the body under the byte and time ceilings, checks the actual digest, and atomically consumes the nonce before parsing the request. A failed signature or body-digest check consumes no nonce. Duplicate authentication or body- digest headers are malformed.

Validators and miners MUST enforce the policy-pinned HTTP body ceilings while streaming, before buffering or parsing the full body. A declared `Content-Length` above the limit is rejected immediately; an absent or false length does not disable the streaming counter. An over-limit miner response becomes one canonical `outer_invalid` record containing only the limit reason and the hash of the bounded prefix retained for evidence. Validators and miners MUST NOT preserve an unbounded body merely to prove that it was oversized.

Protocol and artifact requests MUST advertise `Accept-Encoding:identity`. Receivers MUST reject a response carrying any non-identity `Content-Encoding` and MUST apply byte ceilings to the undecoded response stream. Compression at a storage or transport layer outside the signed HTTP representation does not change this rule.

The policy pins `btauth_max_age_seconds` and `btauth_allowed_skew_seconds`. The maximum age MUST be at least `issue_allowance_seconds`, because the exact initial signed request is committed and finalized before transmission. At launch these values are 360 seconds and 2 seconds. Miners MUST retain replay nonces for at least their sum, and every retry still uses a fresh nonce. Expanding the freshness window does not permit replay of an already accepted signature. Durable replay state MUST be keyed by decoded policy-validator `AccountId32`, reject accounts outside that registry before insertion, and enforce per-validator, total-row, and database-byte ceilings derived from the maximum assignment and retry counts. Reaching a ceiling fails closed and MUST NOT evict a still-fresh nonce.

The policy also pins `maximum_http_header_bytes`, `maximum_request_transmissions_per_assignment`, `maximum_response_bodies_per_assignment`, `maximum_video_fetch_attempts_per_actor`, `maximum_assignment_wire_bytes`, `maximum_validator_window_wire_bytes`, `maximum_audit_bundle_bytes`, `validator_capacity_set_root`, `resource_deadline_stages`, and `resource_utilization_percentile`. Every byte from a successful, failed, truncated, or duplicate attempt counts. Once an assignment ceiling is reached, the validator records one bounded `resource_limit` failure and makes no further attempt. Reaching the validator window ceiling makes that validator skip the window and publish a resource-limit incident bundle. Raw video objects remain separately retained under Section 11 and are not embedded in the audit bundle; their fetch bytes still count toward the wire ceilings.

`maximum_video_fetch_attempts_per_actor` applies to each actor, video hash, and window tuple. A validator's per-assignment counter covers its request and response traffic; its window counter also covers qualification, mirroring, and shared artifact retrieval. A miner's per-assignment counter covers the request it receives, all video-fetch traffic, and the response it sends. A miner isolates unverified video declarations, pending fetches, and failed-fetch budgets by authenticated issuing-validator hotkey. One validator therefore cannot spend another validator's fetch budget or bind a public video digest to false metadata. A hash-verified body may enter the miner's shared cache only after its size and digest match; only that verified body may be reused across validator partitions. The miner's aggregate assignment, wire, and retained-byte ceilings still cover all partitions. Each actor enforces its applicable counter before the next read or write. Shared validator fetches count once at the validator window level rather than once for every miner assignment.

A miner implementation MUST reserve one nonqueueing request-work slot and one runnable inference slot for every validator in the active policy registry. Its total enforced model concurrency MUST be at least that registry size. An isolated model process MUST bind the same minimum, the policy hash, and its model revision to an owner-private startup record that the protocol process checks before each request. These checks do not prove that a model framework or accelerator executes work concurrently internally; the activation soak measures that residual operating risk.

Before the qualifying soak, every validator in the registered soak set signs this RFC 8785 object. The numeric values below illustrate field units; each validator declares its own actual capacities:

```
{
  "schema": "umi-validator-capacity/1",
  "validator_hotkey": "ss58-validator-hotkey",
  "hardware_class": "policy-cost-schedule-class",
  "region_class": "policy-cost-schedule-region",
  "meter_adapter_version": "version-and-content-hash",
  "capacities": {
    "cpu_core_milliseconds_per_window": 1000000,
    "accelerator_milliseconds_per_window": 0,
    "peak_host_memory_bytes": 68719476736,
    "peak_accelerator_memory_bytes": 0,
    "retained_storage_bytes": 2147483648
  }
}
```

The validator signs `SHA256("umi-validator-capacity-v1\0"||RFC8785(capacity_statement))` under Section 6.3. The shadow policy embeds the root of the complete statement set:

```
validator_capacity_leaf = SHA256(
  "umi-validator-capacity-leaf-v1\0" ||
  validator_AccountId32 ||
  SHA256(RFC8785(capacity_statement))
)

validator_capacity_set_root = SHA256(
  "umi-validator-capacity-set-v1\0" ||
  U32BE(validator_count) ||
  concat(leaves_sorted_by_validator_AccountId32)
)
```

The set contains exactly one valid signed statement for every registered soak validator and no other statement. Changing a validator, hardware or region class, meter adapter, or capacity changes the root and the activation-equivalence digest, so it restarts the soak. A zero accelerator capacity declares that no accelerator may be used; otherwise every declared capacity is a positive integer in the unit named by its field.

The cost schedule maps each hardware and region class to independently published core count, accelerator count, host and accelerator memory, and provisioned storage. Declared compute capacity MUST NOT exceed the relevant device count multiplied by `window_stride_blocks*target_block_interval_seconds*1000`; declared memory MUST NOT exceed the class specification, and declared retained storage MUST equal a priced provisioned volume. A lower declaration remains charged at the full selected class or volume, so lowering a denominator cannot reduce the reported cost.

Resource accounting is common across implementations:

- The policy pins an ordered `resource_deadline_stages` list. Each entry names a schedule-derived start, deadline, and bundle-evidenced completion instant for at least pool qualification, assignment issuance, response anchoring, reveal and weight build, and audit release. It also pins the common integer-millisecond timebase and exact extraction rule for all three instants. A stage definition is conforming only when `deadline>start`. Success requires `start<=completion<deadline`, and its utilization is `(completion-start)/(deadline-start)`. A missing or pre-start completion, or completion at or after the deadline, is a capacity deadline miss.
- CPU and accelerator utilization are the window's metered core-milliseconds and active-device milliseconds divided by their declared capacities. Host and accelerator memory utilization use the greatest policy-pinned meter reading in the window divided by the corresponding byte capacity. The hash-pinned meter adapter specifies the exact operating-system and device counters and emits signed raw records into the bundle. For a declared zero accelerator capacity, utilization is zero only when both accelerator meters remain zero; any positive reading fails the gate.
- Transfer utilization is the validator's Section 6.1 window byte count divided by `maximum_validator_window_wire_bytes`. Retained-storage utilization is the peak sum of the uncompressed byte lengths of distinct content-addressed objects the validator is required to hold at that instant, including raw video, divided by `retained_storage_bytes`. Each digest is counted once. Audit-bundle utilization uses the exact byte formula in Section 12 divided by `maximum_audit_bundle_bytes`.
- Every registered soak validator reports every dimension for every member of the complete scheduled-window set in Section 14. A missing statement, raw meter record, or dimension fails the gate. Percentiles are never pooled across validators. For each `(validator_hotkey, non_deadline_dimension)` and each `(validator_hotkey, deadline_stage_id)` separately, take exactly one ratio from every scheduled soak window, so `n` equals the size of the complete scheduled-window set. Sort those `n` ratios in ascending order and select nearest-rank `resource_utilization_percentile`, at rank `ceil(p*n)`. Every resulting percentile must pass; no window, stage, validator, or sample may be dropped or added.

These records make utilization arithmetic reproducible; they do not make physical meters trustless. Their off-chain status remains a residual assumption in Section 4.7 and their declared hardware cost is tested by Section 4.6.

6.2 Request

```
{
  "protocol": "umi-asl/0.1",
  "window_id": "hex-encoded-sha256",
```

```
  "batch_id": "base64url-128-bit-id",
  "challenge_id": "base64url-128-bit-id",
  "issued_block": 123456,
  "issued_block_hash": "0x...",
  "deadline_block": 123466,
  "response_close_round": 12345578,
  "reveal_round": 12345678,
  "video": {
    "url": "https://delivery.example/v1/umi/deliveries/qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq",
    "sha256": "hex-encoded-sha256",
    "size_bytes": 1234567,
    "media_type": "video/mp4"
  },
  "task": {
    "source_language": "ase",
    "target_language": "en",
    "stratum": "short_utterance"
  },
  "scoring_policy_hash": "hex-encoded-sha256"
}
```

Requirements:

- `batch_id` and `challenge_id` MUST be opaque random values with at least 128 bits of entropy.
- `deadline_block` MUST equal `issued_block+response_deadline_blocks` under the committed window policy.
- The window ID and both rounds MUST match the committed public manifest.
- The URL MUST be a credential-free HTTPS delivery URL issued after batch selection. Its origin MUST appear in the mirror rule's sorted `delivery_origins`, which are disjoint from the authenticated retrieval origins. The URL MUST remain valid through `response_close_round`, expire within the policy-pinned delivery grace after it, and use exactly `/v1/umi/deliveries/<token>`, where `token` is the canonical unpadded base64url encoding of the 24-byte value derived below. Credentials, queries, fragments, percent encoding, additional path segments, and any label, prompt, signer name, or task answer are forbidden. An authenticated mirror object URL MUST NOT enter a miner request.
- Before the first issuance attempt, the validator MUST generate and durably store one 256-bit cryptographically random `delivery_token_seed` for the window. The seed enters the canonical authenticated video-delivery issuance request and MUST be reused after a retry or restart. For each selected item, the delivery service MUST use the first 24 bytes of this digest as the URL token, encoded as canonical unpadded base64url:

```
SHA256(
  "umi-video-delivery-token-v1\0" ||
  base64url_decode(delivery_token_seed) ||
  window_id ||
  U64BE(window_index) ||
  scoring_policy_hash ||
  U64BE(response_close_round) ||
  base64url_decode(batch_id) ||
  base64url_decode(challenge_id) ||
  video_sha256 ||
  U64BE(size_bytes)
)[0:24]
```

Hash fields in this formula are raw 32-byte values. A service-chosen token or a response that does not reproduce this token is invalid and cannot be cached as a successful issuance.

- The validator MUST stream every video fetch through the declared `size_bytes`, `maximum_clip_size`, attempt, assignment, and window counters, abort on the first exceeded ceiling, and verify the completed bytes against `video.sha256` before issuing the request.
- A miner MUST apply the same streaming size and fetch-attempt limits before inference, verify the downloaded bytes, and echo the digest in its response.
- The task and policy hash MUST match the committed batch manifest.

6.3 Response

A successful decrypted response has this logical shape:

```
{
  "schema": "umi-response-plaintext/1",
  "protocol": "umi-asl/0.1",
  "window_id": "hex-encoded-sha256",
  "batch_id": "base64url-128-bit-id",
  "challenge_id": "base64url-128-bit-id",
  "request_digest": "hex-encoded-sha256",
  "issued_block_hash": "0x...",
  "validator_hotkey": "ss58-issuing-validator-hotkey",
  "serving_hotkey": "ss58-serving-miner-hotkey",
  "status": "ok",
  "received_video_sha256": "hex-encoded-sha256",
  "hypothesis": "english text",
  "model_revision": "optional-opaque-sha256",
  "error_code": null
}
```

For `status:"error"`, `hypothesis` and `model_revision` are null, `error_code` is one policy-enumerated value, and `received_video_sha256` is either the verified digest or null when download failed. The miner RFC 8785 encodes this plaintext and encrypts it to the request's ground-truth `reveal_round`. It returns this wire envelope:

```
{
  "schema": "umi-response-envelope/1",
  "protocol": "umi-asl/0.1",
  "window_id": "hex-encoded-sha256",
  "batch_id": "base64url-128-bit-id",
  "challenge_id": "base64url-128-bit-id",
  "request_digest": "hex-encoded-sha256",
  "issued_block_hash": "0x...",
  "validator_hotkey": "ss58-issuing-validator-hotkey",
  "serving_hotkey": "ss58-serving-miner-hotkey",
  "response_tle_profile": "umi-response-tle/1",
  "response_reveal_round": 12345678,
  "encrypted_response": "base64url-scale-userdata",
  "encrypted_response_sha256": "hex-encoded-sha256",
  "signature_scheme": "sr25519"
}
```

`encrypted_response` is unpadded base64url for the exact `umi-tle/1` portable `UserData` bytes. Its embedded round and `response_reveal_round` MUST both equal the request's `reveal_round`, and `encrypted_response_sha256` MUST equal SHA-256 of those decoded bytes. The request and signed-envelope digests are:

```
request_digest = SHA256("umi-request-v1\0" || RFC8785(request))
response_digest = SHA256(
  "umi-response-envelope-v1\0" || RFC8785(wire_envelope_without_signature)
)
```

The validator records the serving hotkey and signature separately from the response envelope. `signature_scheme` MUST be `sr25519` or `ed25519` and follows the exact verification rules of `btauth/1`: sr25519 uses the Substrate signing context, ed25519 uses RFC 8032, and a verifier MUST NOT try both. The signature is 64 bytes, encoded as `0x` plus lowercase hexadecimal, over the raw 32-byte response digest. Hex-encoded digest fields are decoded to their raw 32 bytes in every binary hash formula.

`umi-response-tle/1` uses the same strict SCALE decoder, compressed `TLECiphertext<TinyBLS381>`, Quicknet tuple, and pinned test vectors as `umi-tle/1` in Section 8.2. It changes only the plaintext schema and required reveal round.

6.4 Response rules

- The first structurally valid signed envelope received no later than `deadline_block` and before publication of the `response_close_round` pulse is final. The earlier boundary controls.
- Before that deadline, the validator verifies the outer schema, signature, ciphertext hash, strict portable-envelope decoding, compressed ciphertext structure, and both round copies without decrypting the answer.
- A missing, late, malformed, unsigned, digest-mismatched, undecryptable, or plaintext-mismatched response scores zero. Miner response failure never voids another assignment or the selection window.
- A miner MUST NOT send a readable hypothesis before ground-truth reveal. The validator MUST NOT decrypt any response before `reveal_round`. Once that pulse is public, it decrypts each anchored final envelope and accepts only the exact RFC 8785 `umi-response-plaintext/1` object.
- Every inner protocol, window, batch, challenge, request, block, validator, and serving-hotkey value MUST match the signed envelope and request.
- A transport retry MUST reuse the same challenge ID. A validator MUST NOT exceed `maximum_request_transmissions_per_assignment` or retry after receiving a valid envelope. A miner sends at most one response body per authenticated request transmission. The validator reads at most `maximum_response_bodies_per_assignment`; later bodies are neither parsed nor retained beyond one bounded excess-attempt receipt.
- An `ok` hypothesis over `maximum_hypothesis_utf8_bytes`, `maximum_hypothesis_tokens`, or `maximum_hypothesis_graphemes` scores zero. The grapheme count uses Section 9.1 normalization, removes whitespace, and then uses the policy-pinned segmentation. A valid `error` response scores zero and remains an authenticated miner response.
- `model_revision` is optional and carries zero score weight.
- The encrypted plaintext and signed envelope MUST name the issuing validator's hotkey. A signed response is admissible evidence only in that validator's bundle; this binds every scored response to the validator that actually issued the challenge (Section 9.8).
- `request_digest` MUST match the exact canonical request authenticated by `btauth/1`, and the response signature MUST verify against the serving hotkey snapshotted at the closing block under the declared scheme.
- A miner MUST return an encrypted explicit error status if it cannot fetch or decode the exact video. Synthetic outputs and placeholder translations are invalid.

7 Challenge eligibility

An emission-bearing challenge MUST satisfy every requirement in this section.

7.1 Media profile

- MP4 container with H.264 video and no audio track;
- duration from 2 through 15 seconds;
- maximum 1280 by 720 pixels, 30 frames per second, and 16 MiB;
- stable view of the signing space, including face, torso, and the hands used by the signer;
- metadata stripped before hashing and delivery;
- successful decode by the protocol conformance decoder.

The quality check MAY use pose extraction as one signal. It MUST support valid one-handed signs and MUST fail closed when its required model is unavailable.

The protocol frame digest uses the policy-pinned decoder's RGB24 output in presentation order:

```
frame_digest = SHA256(
  "umi-frames-v1\0" ||
  U32BE(frame_count) ||
  concat(SHA256(U32BE(width) || U32BE(height) || RGB24(frame_i)))
)
```

Decoder, color-conversion, and ordering fixtures MUST match bit-for-bit.

7.2 Linguistic strata

Each batch uses this target mix:

Stratum	Share	Metric
Fingerspelling and numbers	15%	CER over best reference
Short everyday utterances	35%	WER over best reference
Continuous everyday signing	50%	WER over best reference

At the version 0.1 batch size of 12 emission-bearing clips, the allocation is 2 fingerspelling, 4 short-utterance, and 6 continuous clips. Canary items are added outside this allocation. A different batch size requires a policy-pinned largest- remainder allocation with ties resolved in the table order above. Domain-specific and high-consequence material is ineligible in version 0.1.

7.3 References

- A clip MUST have from three through five accepted English references.
- Each accepted reference's raw UTF-8 encoding MUST fit `maximum_reference_utf8_bytes`, and its canonical normalization in Section 9.1 MUST fit both `maximum_reference_tokens` and `maximum_reference_graphemes`. The grapheme count excludes whitespace after normalization.
- References MUST be fixed before the batch commitment.
- At least three independent ASL-fluent reviewers, including at least two Deaf reviewers, MUST first view the clip without the original prompt and submit a locked reconstruction.
- The prompt is shown only after those reconstructions are locked. At least two reviewers MUST then confirm that the clip conveys the prompted meaning.
- An accepted blind reconstruction MAY enter the reference set after duplicate and quality review.
- One contributor or reviewer MUST NOT approve their own work.
- The public ground-truth payload MUST preserve reference ordering.

References describe acceptable English renderings. Loose paraphrases that change meaning are ineligible. For a canary, these requirements apply to the actual references committed in its canary-evidence object; the deliberately mismatched scoring references follow Section 7.6.

7.4 Freshness and diversity

- A script group MUST receive emission-bearing evaluation in one reveal cohort only.
- Every clip in that script group MUST retire when the cohort reveals.
- Exact video hashes, protocol frame digests, and revealed normalized script hashes MUST be checked against the spent registry.
- The full 14-item launch batch, including canaries, MUST contain at least seven signers, with no signer supplying more than two items. The general cap is 20%.
- A rolling four-batch scoring window MUST contain at least two publisher control groups. One control group MUST NOT supply more than 50% of scored clips in that window or more than 50% of any miner's assigned clips.
- Each selection window uses two equal-size batches from distinct control groups and assigns both to the same miner panel. This makes the group cap structural rather than a post-score exclusion. Every assignment that is issued remains in the score denominator.

Several signer variants for one script MAY appear in the same sealed cohort and retire together. A selected two-batch window MUST contain at most one variant of a script. A duplicate discovered after reveal makes the window void.

7.5 Consent and provenance

The publisher MUST hold a signed consent record that covers benchmark delivery to independent network participants, scoring, audit retention, and the limits of deletion after distribution. Training, public release, and product use require separate permission flags.

Emission-bearing clips MUST have a provenance manifest and MUST exclude minors in version 0.1. A consent or rights failure found before issuance invalidates the anchored batch, so the validator skips the selection window. If discovered after issuance, it makes that window void.

7.6 Canary items

Let `N` be the number of emission-bearing clips in a batch and `f_canary` the declared canary fraction. The publisher adds this many canaries beyond `N`:

```
C = max(1, ceil(N * f_canary))
```

At the version 0.1 values, `N=12` and `f_canary=0.10`, so a batch contains 12 emission-bearing clips plus two canaries. One canary uses CER and one uses WER; the WER canary uses short utterance when the least-significant bit of the final digest byte of `SHA256("umi-canary-stratum-v1\0"||window_id||base64url_decode(batch_id))` is zero and continuous otherwise. Each canary pairs a real eligible clip with a committed reference set from a different reserved script group. Canaries are indistinguishable from scored items before reveal. After reveal they are marked `canary:true`, carry zero score weight, and are absent from every score denominator. The delivered clip, its actual script group, and the reserved reference script group all retire with the cohort.

A canary pair is eligible only if every normalized actual reference scores below 0.10 against every mismatched reference under the canonical clip metric. Fixed rational CER and WER hit thresholds are selected on development data and then frozen. On an independent no-leak confirmation set, each metric MUST record zero hits in at least 40,000 honest comparisons. This gives a one-sided 95% Clopper-Pearson upper bound below 0.000075 per comparison and, by the union bound, an honest-window false-void bound below 0.01 at 128 comparisons. Each metric MUST also detect at least 99% of 1,000 injected-reference leaks, with a one-sided 95% lower bound of at least 0.99. Comparisons use pinned integer or rational arithmetic. Changing the construction rule or either threshold requires a new scoring policy hash and a new confirmation set.

The encrypted ground-truth item contains a canary-evidence record with the actual references, actual script hash, reserved script hash, and mismatched references. It becomes public with the rest of the payload after reveal. A missing or inconsistent record makes the selection window void.

A hypothesis that scores at or above the declared metric-specific canary threshold against its mismatched reference set is evidence that sealed reference text may have left the intended channel. A hit voids the affected selection window, appears with full evidence in the audit bundle, and triggers the incident procedure. A hit alone does not identify the leaking party or exclude a publisher or miner. When translation weights are active, a hit also pauses new UMI weight submissions until the public incident procedure resolves the affected delivery and reveal path. Repeated hits remain unattributed incidents; they do not create publisher strikes without deterministic evidence that identifies a publisher fault.

Canaries can detect leakage by infrastructure or by a party that does not know the canary assignment. They cannot detect a publisher selectively leaking its own live references, because the publisher knows which items are canaries. Control-group caps, locked collateral, public audit, and the shadow monitoring in Section 9.8 bound that residual risk.

8 Batch lifecycle

8.1 Prepare

The scoring policy defines the window clock. It pins `activation_block`, `window_stride_blocks`, `proposal_blocks`, `anchor_blocks`, `target_block_interval_seconds`, `selection_finality_buffer_seconds`, `issue_allowance_seconds`, `response_window_seconds`, `delivery_grace_seconds`, and `reveal_margin_seconds`. The same policy pins `weight_commit_buffer_blocks` for the post-reveal sequence in Section 10.3. For zero-based window index `j`, every actor derives:

```
announcement_block = activation_block + j * window_stride_blocks
proposal_close_block = announcement_block + proposal_blocks
closing_block      = announcement_block + anchor_blocks

selection_round = RoundAtMs(
  timestamp_ms(announcement_block) +
  1000 * (
    anchor_blocks * target_block_interval_seconds +
    selection_finality_buffer_seconds
  )
)

issue_close_round = selection_round + ceil(issue_allowance_seconds / 3)
response_close_round = issue_close_round + ceil(response_window_seconds / 3)
response_deadline_blocks =
  ceil(
    (issue_allowance_seconds + response_window_seconds) /
    target_block_interval_seconds
  )

reveal_round = response_close_round + ceil(reveal_margin_seconds / 3)
```

`RoundAtMs` returns the first round of the pinned three-second Quicknet at or after the supplied UTC millisecond. The announcement block MUST be finalized before a publisher prepares its window artifacts. The policy MUST set `0<proposal_blocks<anchor_blocks`, leaving a certification interval before anchor close. The closing block MUST be finalized before `selection_round` is published. A new window opens only after the preceding window's `reveal_round` and successful spent-state transition. Failure of any condition makes that window void.

The runtime target block interval at policy activation MUST equal the pinned value. A runtime change requires a new policy hash. Every candidate batch in a window uses that window's exact `response_close_round` and `reveal_round`; batches cannot carry into another window. The window identifier is:

```
window_id = SHA256(
  "umi-window-v1\0" ||
  U16BE(netuid) ||
  U64BE(j) ||
  announcement_block_hash ||
  U64BE(closing_block) ||
  U64BE(selection_round) ||
  U64BE(response_close_round) ||
  U64BE(reveal_round) ||
  scoring_policy_hash
)
```

Hash values in this section are raw 32-byte values. Unsigned integers use the fixed-width big-endian encoding named in each formula. Text outside canonical JSON is UTF-8. A name ending in `_AccountId32` means the 32 account bytes obtained by decoding its SS58 address.

A publisher prepares one public manifest and one ground-truth payload per batch. The public manifest contains the window and batch IDs, clip video hashes, protocol frame digests, strata, media properties, publisher hotkey, scoring policy hash, timelock profile, response-close round, reveal round, and ciphertext hash. It contains no prompt or reference text.

Pool bodies, final pool manifests, public batch manifests, and portable ground-truth envelopes MUST fit their policy-pinned byte ceilings before parsing. Artifact retrieval uses a streaming counter and aborts at the ceiling. An over-limit candidate is ineligible before certification; a certificate signer MUST NOT sign a set containing it.

The ground-truth payload has this logical shape:

```
{
  "schema": "umi-ground-truth/1",
  "window_id": "...",
  "batch_id": "...",
  "scoring_policy_hash": "...",
  "tle_profile": "umi-tle/1",
```

```
  "response_close_round": 12345578,
  "reveal_round": 12345678,
  "items": [
    {
      "challenge_id": "...",
      "metric": "wer",
      "canary": false,
      "references": ["reference one", "reference two", "reference three"],
      "canary_evidence": null,
      "normalized_script_sha256": "...",
      "retirement_script_sha256s": ["..."],
      "consent_manifest_sha256": "..."
    }
  ]
}
```

`normalized_script_sha256` is not a hash of the publisher's raw prompt bytes. For ordinary, canary-actual, and canary-reserved scripts alike, the publisher applies the complete Section 9.1 text normalization and hashes the normalized UTF-8 bytes:

```
normalized_script_sha256 = SHA256(UTF8(normalize_text_section_9_1(script)))
```

The JSON field carries the 32-byte result as lowercase hexadecimal. A script whose normalized form is empty is ineligible.

For an ordinary item, `retirement_script_sha256s` contains its normalized script hash and `canary_evidence` is null. For a canary, `canary_evidence` contains the actual references, actual script hash, reserved script hash, and mismatched references; `retirement_script_sha256s` contains the sorted unique script hashes. Batch items are ordered by `challenge_id`.

Each publisher then creates one canonical pool manifest for the window:

```
{
  "schema": "umi-pool-manifest/1",
  "window_id": "...",
  "publisher_hotkey": "...",
  "scoring_policy_hash": "...",
  "batches": [
    {
      "batch_id": "...",
      "batch_commitment": "...",
      "public_manifest_sha256": "...",
      "ciphertext_sha256": "...",
      "reveal_round": 12345678
    }
  ],
  "availability_certificate": {
    "schema": "umi-availability/1",
    "availability_set_root": "...",
    "qualified_pool_leaves": ["..."],
    "signatures": [
      {"validator_hotkey": "...", "scheme": "sr25519", "signature": "..."}
    ]
  }
}
```

Batch entries are sorted lexicographically by decoded `batch_id`, and duplicate batch IDs or commitments make the pool manifest invalid. A publisher releases its pool body before `proposal_close_block`; the pool body is the RFC 8785 object above with `availability_certificate` omitted. Validators derive:

```
availability_leaf = SHA256(
  "umi-availability-leaf-v1\0" ||
  publisher_AccountId32 ||
  SHA256(RFC8785(pool_body))
)
```

```
availability_set_root = SHA256(
  "umi-availability-set-v1\0" ||
  U32BE(qualified_pool_count) ||
  concat(sort_lexicographically(qualified_pool_leaves))
)

availability_digest = SHA256(
  "umi-availability-v1\0" ||
  window_id ||
  availability_set_root
)
```

Each certificate signature follows the response-signature encoding and scheme rules in Section 6.3 and covers the raw 32-byte `availability_digest`. The signer account and validator permit are resolved at the finalized announcement block.

The scoring policy contains an ordered validator registry. For a window, its active subset contains the listed hotkeys that hold a validator permit at the finalized announcement block. Let `V` be that subset's size and `q_availability=max(3,floor(2*V/3)+1)`; fewer than four active validators makes the window void. This allows one missing signature at the launch minimum. A validator qualifies a pool leaf only after retrieving and hash-checking all of its listed public manifests, ciphertexts, and videos and rejecting public video or frame hashes already spent or duplicated in the proposed set. It also decodes each portable timelock envelope, rejects trailing bytes, verifies the compressed ciphertext structure, and checks every public reveal-round copy. Before `closing_block`, it signs at most one `availability_digest` for the window, retains every artifact covered by that set through reveal, and serves them to active validators through the policy's authenticated mirror rule, subject to Section 11.

A valid certificate contains the complete sorted leaf set and at least `q_availability` signatures over one root, sorted by validator account. Every final pool manifest carries that same certificate, and its recomputed leaf MUST appear in the set. The set contains at most one leaf per active publisher. Conflicting quorum certificates make the window void. A validator that cannot retrieve one valid quorum certificate skips the window; it MUST NOT infer a smaller availability set. An uncertified pool is ineligible and cannot halt the window.

The active policy sets `max_active_publishers`, `max_active_control_groups`, `max_candidate_batches_per_publisher`, `max_candidate_batches_per_group`, and `max_candidate_batches_total`. At launch the registry contains exactly three active publisher hotkeys in exactly three control groups, each group may contribute at most one batch, and the entire window may contain at most three candidate batches. A pool body or availability certificate that exceeds any limit is invalid. The policy also defines the content-addressed mirror discovery rule. Its canonical bytes contain sorted authenticated retrieval origins, disjoint sorted public `delivery_origins`, and the fixed authenticated post-selection delivery-issuance endpoint. Publisher and validator registry changes require a new policy hash and activation block.

8.2 Seal, certify, and anchor

`umi-tle/1` uses the current Bittensor timelock portable format. It is the exact SCALE encoding of `UserData{encrypted_data:Vec<u8>,reveal_round:u64}`, with no trailing bytes. `encrypted_data` is the canonical compressed `TLECiphertext<TinyBLS381>`. Every duplicated reveal-round value in the portable envelope, public manifest, ground-truth metadata, or pool manifest MUST match. The profile pins this drand network tuple:

```
beacon_id    = quicknet
chain_hash   = 52db9ba70e0cc0f6eaf7803dd07447a1f5477735fd3f661792ba94600c84e971
scheme_id    = bls-unchained-g1-rfc9380
period       = 3
genesis_time = 1692803367
public_key   = 83cf0f2896adee7eb8b5f01fcad3912212c437e0073e911fb90022d3e760183c8c4b450b6a0a6c3ac6a5776a2d1
064510d1fec758c921cc22b0e17e63aaf4bcb5ed66304de9cf809bd274ca73bab4af5a6e9c76a4bc09e76eae8991ef5ece45a
```

The scoring policy pins the Bittensor timelock package by version and content hash, plus a portable-envelope test vector. At policy activation, validators compare the pinned tuple with `Drand.BeaconConfig` at the activation block. A tuple, decoder, or test-vector mismatch fails closed and requires a new policy version.

The publisher encrypts the RFC 8785 canonical ground-truth bytes to the declared future round. Before `proposal_close_block`, it publishes every public manifest and ciphertext and makes each video available through the authenticated mirror policy. The availability signers become at least three independent mirrors before the publisher anchors its final pool manifest. Every participating validator retrieves and hash-checks the certified pool manifests, public batch manifests, and ciphertexts. After selection, it retrieves every selected video, then sends one canonical authenticated request to the mirror rule's fixed delivery-issuance endpoint before preparing miner assignments. The request binds the window, batch, challenge, video hash, and size. The canonical response binds the same fields and provides one credential-free HTTPS URL and expiry per selected item. The validator accepts only URLs under `delivery_origins` whose expiry is at least the response close time and at most the end of the delivery grace. The exact discovery rule, request, response, and non-secret exchange evidence enter the audit bundle and are replayed before accepting the assignments. Every publisher, availability signer, validator, and miner fetch streams HTTP headers and bodies through the applicable header, object-size, fetch-attempt, assignment, and validator-window counters in Section 6.1. It aborts the transfer at the first exceeded ceiling and retains only the bounded receipt. An availability signer MUST NOT qualify a pool whose complete artifact set it could not retrieve within those ceilings. Local unavailability never removes one batch from the pool; a validator that lacks a certified artifact skips the whole window and emits a certificate-breach alert.

A breached certificate never salvages or shrinks the current window. The signed promise, requested hashes, and retrieval records enter the incident bundle. The next scheduled window runs a fresh qualification. Material from the failed pool that crossed the external-disclosure boundary below is ineligible for that or any later proposal, even if no quorum formed or no pool anchor finalized. Removing a publisher or availability signer from the active registry still requires a new policy hash and cannot change the failed window.

The first time a publisher gives any window-bound pool body, public manifest, ground-truth ciphertext, challenge video, or retrieval access to a validator, mirror, public store, or any other actor outside its control group, it MUST append one permanent record to its private disclosure-quarantine ledger. The record MUST bind the window ID, batch ID, batch commitment, reveal round, first-disclosure time, recipient class, disclosure-evidence digest, all 14 video SHA-256 values, all 14 frame digests, and all 16 normalized script hashes: 14 actual scripts and the two reserved canary scripts. `first-disclosuretime` is the instant the first external access authorization becomes valid; `disclosure-evidencedigest` binds the signed authorization or transfer receipt prepared for that boundary. The publisher MUST exclude every recorded video, frame, and script hash from all later pool bodies, whether qualification succeeds, a quorum forms, certification completes, or anchoring succeeds. A release built and retained entirely inside the publisher control group does not cross this boundary; it MAY be reused only after verified deletion of every window-bound artifact and access credential.

The envelope reveal round MUST equal the window-derived value in Section 8.1. A candidate batch belongs to that window only. When its reveal round arrives, an unselected batch retires unused: its references become public, its script group is spent, and its clips are permanently ineligible for later UMI scoring. Publishers SHOULD size sealed inventory to one window; unselected expiry is a publisher cost and never a scoring input. The launch cap limits this expiry to one unselected 14-item batch per valid window. `Spent` burns benchmark eligibility, not the underlying data or permissions granted by its consent class.

Each batch commitment is:

```
batch_commitment = SHA256(
  "umi-batch-v1\0" ||
  RFC8785(public_manifest) ||
  SHA256(ciphertext) ||
  U64BE(reveal_round)
)
```

Before the closing block, the publisher signs `Commitments.set_commitment` with its registered hotkey. The commitment contains exactly one `Data::Sha256(SHA256(RFC8785(pool_manifest)))` field. The publisher MUST NOT replace that live record until the closing block is finalized and its storage proof has been retained. Because the pallet stores one current record per `(netuid, hotkey)`, pool-anchor windows do not overlap. Finalized transaction inclusion and the retained closing-block storage proof establish the exact closing state even after live commitment storage is replaced or purged.

8.3 Select

At the finalized closing block, validators read the active publisher registry, registration and ownership state, collateral state, and each publisher's live pool anchor from that one block. An eligible anchor must have been included no later than the closing block, resolve to a valid pool manifest for the declared window and policy, carry the unique quorum certificate, and have its recomputed availability leaf in that certificate's set. Its control group MUST be eligible under the publisher-fault state derived through the preceding reveal cohort. Validators verify its RFC 8785 hash, entry ordering and limits, then recompute every public-manifest hash, ciphertext hash, and batch commitment. A missing, malformed, or uncertified anchor contributes no candidates and cannot halt the window. Failure to retrieve an artifact covered by the valid certificate causes the validator to skip the window; it MUST NOT construct a smaller local pool.

For every eligible batch, define:

```
pool_leaf = SHA256(
  "umi-pool-leaf-v1\0" ||
  publisher_AccountId32 ||
  batch_commitment
)

candidate_pool_root = SHA256(
  "umi-pool-root-v1\0" ||
  U32BE(pool_leaf_count) ||
  concat(sort_lexicographically(pool_leaves))
)
```

The candidate miner set is snapshotted from the metagraph at the same finalized closing block. It contains registered hotkeys with a syntactically valid serving record, excluding active publisher hotkeys and hotkeys with a validator permit. Reachability is never a preselection filter; an unavailable selected miner receives assigned zeros. The common selection seed and each batch rank are:

```
selection_seed = SHA256(
  "umi-select-v2\0" ||
  drand_signature(selection_round) ||
  candidate_pool_root
)

batch_rank = SHA256(
  "umi-batch-rank-v1\0" ||
  selection_seed ||
  pool_leaf
)
```

Here `drand_signature` means the verified raw signature bytes, not its hexadecimal text. The signature and verification evidence enter the audit bundle. `selection_round` is declared before pool anchoring. The closing block MUST reach finality before that round's pulse is published; otherwise the entire window is void. The validator proves that ordering with the closing header's hash-chained local acceptance receipt. Its timestamp is evidence of that validator's local clock, not chain- or GRANDPA-certified time, and cannot constrain a dishonest validator clock. A block hash is not used as entropy: block producers influence block hashes, and a one-block grind at pool close would bias the entire cohort's selection. The drand signature of a future round is unpredictable to every chain participant, and the pool root and miner snapshot are fixed before that round exists, so neither publishers, miners, nor block producers can grind membership toward a favorable draw. Validators sort by `(batch_rank, pool_leaf)` and take the declared batch count, skipping any batch whose publisher control group was already selected. A window with too few distinct control groups is void.

For each candidate miner hotkey, the validator computes:

```
miner_rank = SHA256(
  "umi-miner-rank-v1\0" ||
  selection_seed ||
  validator_AccountId32 ||
  miner_AccountId32
)
```

Let `target_count` be the smaller of the policy-pinned miner-panel size and the candidate-miner count. Each validator forms a panel of exactly `target_count` miners for the selection window and assigns every item in every selected batch, including canaries, to every panel member. It reserves `ceil(0.20*target_count)` exploration positions for eligible miners with the fewest assigned observations, resolving count ties by `miner_rank`. For this comparison, `assigned_observation_count_v(root)` is the number of requests that validator `v` issued to that miner root in all earlier windows under the active policy, including requests from a window later declared void. Counts are derived from the pre-reveal request-set anchors in Section 8.4. A chain-recorded successor inherits its root's count; a previously unseen root starts at zero. It fills the remaining positions by `miner_rank` from hotkeys not already selected. Current score never affects selection. Exploration confers evaluation, not weight: a hotkey earns weight only after meeting the observation minimum in Section 9.5. Registration cost raises the cost of hotkey churn aimed at exploration positions.

The validator-bound rank changes panel membership only when the candidate-miner count exceeds `target_count`. In that case, validators normally hold overlapping but non-identical samples. When every candidate fits in the panel, every validator selects the full candidate set and this sampling rule provides no cross-validator diversity. Let `w_v` and `w_u` be two validators' normalized pre-quantization vectors over the full UID set, and let `k` equal the live `MinAllowedWeights`:

```
TV(v, u) = 0.5 * sum_i(abs(w_v[i] - w_u[i]))
top_k_overlap(v, u) = size(top_k(w_v) intersect top_k(w_u)) / k
```

During the ten-tempo shadow trial, every tempo MUST have median pairwise `TV` at most 0.10, maximum pairwise `TV` at most 0.20, and `top_k_overlap` at least 0.80 for every validator pair. Ties in `top_k` resolve by decoded miner account. Failure requires a larger panel, longer rolling window, or another declared sampling change before UMI weight activation. A zero distance caused by full-panel coverage measures agreement, but it is not evidence of distinct sampling.

8.4 Serve and respond

The validator first constructs the exact initial signed request for every deterministic assignment. Let `initial_auth_record` be the canonical authentication record for that first transmission. Before sending any request, it derives:

```
assignment_leaf = SHA256(
  "umi-assignment-leaf-v1\0" ||
  miner_AccountId32 ||
  request_digest ||
  SHA256(RFC8785(initial_auth_record))
)

assignment_set_root = SHA256(
  "umi-assignment-set-v1\0" ||
  window_id ||
  validator_AccountId32 ||
  U32BE(assignment_leaf_count) ||
  concat(sort_lexicographically(assignment_leaves))
)
```

There MUST be exactly one leaf for every selected batch item and panel-member pair, including canaries, and no other leaf. The validator publishes exactly one `Data::Sha256(assignment_set_root)` field with `Commitments.set_commitment` and retains finalized inclusion and storage proofs. Only then may it transmit those exact initial signed requests. Every initial request MUST be sent before `issue_close_round`. A missing, late, or unfinalized assignment-set anchor, a different transmitted request, or incomplete issuance makes that validator skip its weight update and publish an issuance-failure alert. The anchor proves that the complete request set existed before issuance; it does not prove network delivery.

Before publishing the assignment-set anchor, the validator computes the worst-case wire, retained-storage, and audit-bundle bytes implied by the assignment count, object cardinalities, and the policy's body, header, attempt, and object-size ceilings. It uses the accounting rules in Sections 6.1 and 12. If any bound exceeds `maximum_validator_window_wire_bytes`, the validator's signed `retained_storage_bytes`, or `maximum_audit_bundle_bytes`, respectively, it skips the window before issuance and publishes a bounded resource-limit incident. A conforming implementation

cannot rely on compression, deduplication beyond one copy per content digest, or expected miner behavior to pass this preflight.

The validator records each receive time, signed response-envelope bytes, and miner signature. It cannot score a partial panel.

Before `response_close_round`, the validator commits the exact issued set. For each request it forms an RFC 8785 `auth_records` array containing every signed transmission attempt, sorted by numeric nonce. Each record contains the authentication version, scheme, method, wire request target, raw-body SHA-256, nonce, sender, receiver, and signature exactly as sent under `btauth/1`. Every retry uses a fresh nonce and signature. Repeated attempts remain one assignment and one request leaf. Its first record MUST equal the assignment leaf's `initial_auth_record`, and its request digest and miner account MUST equal the assignment leaf. The validator then derives:

```
request_leaf = SHA256(
  "umi-request-leaf-v1\0" ||
  miner_AccountId32 ||
  request_digest ||
  SHA256(RFC8785(auth_records))
)

request_set_root = SHA256(
  "umi-request-set-v1\0" ||
  window_id ||
  validator_AccountId32 ||
  U32BE(request_leaf_count) ||
  concat(sort_lexicographically(request_leaves))
)
```

The request leaves MUST be a bijection with the assignment leaves under miner account and request digest. There can be no added, removed, or substituted request.

The validator publishes exactly one `Data::Sha256(request_set_root)` field with `Commitments.set_commitment`. Its inclusion-block timestamp MUST be earlier than the Quicknet reveal time of `response_close_round`. The validator retains finalized inclusion and storage proofs before replacing its live commitment and MUST NOT send another attempt after this anchor is included. A missing, late, or unfinalized request-set anchor makes that validator skip the window. This anchor proves that the complete signed request set existed before responses closed; it does not prove that a remote miner received every request.

Each response closes at the earlier of its derived block deadline and the common `response_close_round`. The separate `reveal_round` follows after the fixed safety margin. If ground truth becomes available before `response_close_round`, the whole selection window is void.

After the verified response-close pulse and before ground-truth reveal, the validator fixes one `sealed_response_record` for every request leaf without decrypting any response. The policy enumerates its outer disposition codes. A `sealed` record contains the exact wire-envelope hash, signature scheme, serving hotkey, signature, and recorded receipt metadata. A `missing`, `late`, or `outer_invalid` record contains its code, receipt metadata, and the hash of any received bytes. The canonical sealed record produces:

```
response_leaf = SHA256(
  "umi-response-leaf-v1\0" ||
  request_leaf ||
  SHA256(RFC8785(sealed_response_record))
)

response_set_root = SHA256(
  "umi-response-set-v1\0" ||
  window_id ||
  validator_AccountId32 ||
  U32BE(response_leaf_count) ||
  concat(sort_lexicographically(response_leaves))
)
```

There MUST be exactly one sealed response record for every request and no other leaf. The validator replaces its request-set commitment with exactly one `Data::Sha256(response_set_root)` field only after retaining the request-anchor proof. Its inclusion-block timestamp MUST be at least the Quicknet reveal time of `response_close_round` and earlier than the reveal time of `reveal_round`; its inclusion and live storage proof MUST reach finality before `reveal_round` is published. Otherwise that validator skips its weight update. The pre-reveal root prevents responses or missing markers from being invented after ground truth opens. Because the answers remain unreadable until that opening, the validator cannot learn a non-colluding miner's answer and add a copy to the anchored set. Network delivery and the local receipt time of a missing or late response remain validator assertions, bounded by independent validator sampling and consensus rather than cryptographic proof.

8.5 Reveal and score

After `reveal_round` is published, validators decrypt the previously fetched ground-truth ciphertexts and every `sealed` miner envelope, then verify their hashes, signatures, inner bindings, and schemas. A valid decrypted miner `ok` record supplies the hypothesis; a valid miner `error`, failed decryption, invalid plaintext, or non-`sealed` record scores zero. A ground-truth decryption failure makes the entire selection window void. Validators then calculate scores from the responses and references opened together.

8.6 Retire and audit

Batch state is derived from finalized evidence:

1. `proposed`: a pool body is public before `proposal_close_block` but has no protocol eligibility;
2. `anchored-eligible`: after final close, its pool has the quorum availability certificate, a matching eligible chain anchor, valid artifacts, and no prior spent hash;
3. `selected-pending` or `unselected-pending`: the selection pulse fixes which of the anchored candidates receive assignments; both states await the same declared reveal round;
4. `spent`: at the declared reveal round, every candidate retires regardless of selection or successful ground-truth decoding.

A batch commitment MUST occur in exactly one certified pool body and one window. A duplicate across certified pools makes the window void. The next window cannot open before all candidates in the preceding window reach `spent`, so pending content cannot re-enter a later candidate pool.

The disclosure-quarantine ledger is publisher-local operational state, not a fifth canonical batch state and not an input to the spent root, candidate pool, score, or validator weight. A validator MUST NOT remove a candidate or change a result from a private quarantine record or a locally observed failed attempt. Subquorum recipients may retain signed disclosure evidence, but their observations need not be complete or identical. Auditors can prove a disclosed attempt when shown its evidence; they cannot prove that the publisher disclosed every failed attempt. The publisher's private operating evidence MUST record quarantined inventory, including attempts that never became `anchored-eligible`.

The spent registry is derived state. No publisher, validator, API, or database is authorized to append to it directly. For each reveal round, take every unique batch commitment that appeared in an eligible finalized pool manifest and declared that round, whether selected, unselected, scored, void, or undecodable. Create these typed leaves from the commitment, all public hashes, and every successfully revealed retirement script hash:

```
SHA256("umi-spent-batch-v1\0"  || batch_commitment)
SHA256("umi-spent-script-v1\0" || normalized_script_sha256)
SHA256("umi-spent-video-v1\0"  || video_sha256)
SHA256("umi-spent-frame-v1\0"  || frame_digest)
```

Exact video or frame duplicates within the cohort are invalid. Script duplicates within one cohort are permitted only for the signer variants allowed by Section 7.4 and produce one leaf. A hit against any earlier spent leaf makes the new item ineligible; a script hit discovered only after reveal makes its selected window void.

The cohort delta contains the sorted unique leaves absent from the previous registry. Its binary Merkle root uses internal nodes `SHA256("umi-spent-node-v1\0"||left||right)`; the final node at an odd-width level is duplicated. The empty delta is `SHA256("umi-spent-empty-v1\0")`. Registry state begins as 32 zero bytes and advances in reveal-round order:

```
spent_root_r = SHA256(
  "umi-spent-root-v1\0" ||
  spent_root_previous ||
  U64BE(reveal_round) ||
  cohort_delta_root
)
```

Publisher reliability is derived separately from objective reveal evidence. A `publisher_reveal_fault` occurs only when the canonical certified bytes establish one of the policy-pinned reason codes:

- the portable envelope passed pre-close structural checks but cannot decrypt with the verified signature for its declared round;
- the decrypted bytes are not the committed RFC 8785 `umi-ground-truth/1` object;
- a window, batch, policy, round, item-set, or public-hash value in the plaintext disagrees with its committed manifest; or
- a revealed retirement script hash duplicates an earlier spent script.

A canary hit, subjective reference-quality dispute, consent dispute, or validator-local retrieval failure is not a publisher reveal fault. Those cases follow their own incident rules because the public bytes do not identify one publisher fault deterministically.

The policy represents each control-group ID as 32 bytes and assigns every reason a `u16` code. For each attributable `anchored-eligible` batch, derive one leaf and update the rolling root with sorted unique leaves:

```
publisher_fault_leaf = SHA256(
  "umi-publisher-fault-leaf-v1\0" ||
  control_group_id32 ||
  publisher_AccountId32 ||
  window_id ||
  batch_commitment ||
  U16BE(reason_code)
)

publisher_fault_root_j = SHA256(
  "umi-publisher-fault-root-v1\0" ||
  publisher_fault_root_previous ||
  U64BE(j) ||
  U32BE(fault_leaf_count) ||
  concat(sort_lexicographically(publisher_fault_leaves))
)
```

The root begins as 32 zero bytes. At most one strike accrues to a control group in one window even if several reason codes apply. After its first strike, every pool from that group is ineligible for the next `publisher_fault_cooldown_windows` scheduled windows, indices `j+1` through `j+publisher_fault_cooldown_windows`. A second strike under the same protocol version makes the group ineligible for the rest of that version. Reinstatement requires a new protocol version and scoring-policy activation; a new policy hash within version 0.1 is not enough and cannot rewrite the fault history. Because all validators derive this state from the same certified ciphertext, manifest, pulse, and revealed bytes, no validator or owner votes on a strike.

Before committing weights, each validator replays from the last reconciled root and computes the spent and publisher-fault transitions. A local replay failure or missing canonical artifact causes that validator to skip its weight update. A checkpoint is an untrusted optimization and MUST reproduce from finalized pool-anchor proofs and revealed artifacts.

Validators MAY publish the candidate-pool, spent, and publisher-fault roots as soon as ground truth is public because those roots contain no miner outcomes, scores, or weights. A peer root is never authoritative. A mismatch triggers an incident and pauses that validator's subsequent weight updates until independent replay resolves it. The complete transition evidence remains withheld until the audit-release rule in Section 10.3 is satisfied.

If a malformed ciphertext never reveals a valid payload, its public video and frame hashes still retire. Its hidden script hash cannot be recovered, which remains a publisher-trust limitation. If selected, it makes the window void.

The validator computes the spent and publisher-fault transitions, skips the window's weight update, and follows the pre-commit incident-release rule in Section 10.3. Retirement authorizes no additional data use.

9 Deterministic scoring

9.1 Text normalization

For each hypothesis and reference:

1. apply Unicode NFKC;
2. apply Unicode lowercase mapping;
3. tokenize letters and numbers, retaining apostrophes only when surrounded by letters or numbers;
4. replace all other characters with a separator;
5. collapse separators and whitespace;
6. remove leading and trailing whitespace.

The scoring policy pins one canonical open-source scoring package by version and content hash, including its Unicode data version for NFKC and grapheme segmentation. Independent computation in this protocol means independent execution over independently gathered source evidence; it does not require an independent reimplementation of Unicode. Implementations MUST either embed the canonical package or pass its full normalization and segmentation fixture set bit-exactly before they can submit weights. Exact cross-validator reproduction (Section 12) is a property of the pinned package, and a Unicode data update is a scoring-policy version change.

9.2 WER

For token sequences `h` and `r`:

```
WER(h, r) = levenshtein_tokens(h, r) / max(1, token_count(r))
score_wer(h, R) = max over r in R of clamp(1 - WER(h, r), 0, 1)
```

9.3 CER

CER uses the same formula over Unicode grapheme clusters after removing whitespace. The clip score is the best score across its committed references.

All emission-bearing arithmetic before chain encoding uses exact integers and rationals. Decimal values in this document are shorthand for fixed rational numbers. Floating-point arithmetic is non-conforming.

9.4 Assigned failures

Every issued assignment from a valid selection window appears in the denominator. A rejected or missing response has clip score zero. If either selected batch has a protocol-void condition, the full two-batch window contributes no score to any miner. Validators MUST NOT void an individual response or calculate a miner score from successful responses alone.

9.5 Batch and rolling score

Each validator maintains one ordered rolling queue under the active policy. When a valid selection window finishes, its two batches enter in `(batch_rank, pool_leaf)` order. The queue retains the latest `rolling_batch_count` batches globally, not the latest batches assigned separately to each miner. A void or skipped window does not enter or evict a batch. Independently, a batch whose source window index is not in `0<=j-batch_window_index<score_max_age_windows` expires even if void windows prevented the queue from advancing. This prevents an old successful evaluation from remaining live through an extended data outage.

For miner `i` and stratum `k`, let `mean(i, k)` be the arithmetic mean of every assignment to `i` in the retained queue. Unassigned clips create no row. An issued missing or invalid response creates a zero row. The accuracy score is:

```
A_i = 0.15 * mean(i, fingerspelling)
    + 0.35 * mean(i, short_utterance)
    + 0.50 * mean(i, continuous)
```

A miner becomes weight-eligible after at least `minimum_assigned_clips` retained assignments, including at least `minimum_clips_per_stratum` from every stratum. Batches enter this state only as complete valid two-batch selection windows. Every issued assignment in a retained batch counts toward the observation minimum, including a missing or invalid response. If expiration leaves no eligible miners, the validator follows the skip rule in Section 9.6. These transitions are local to the validator because requests and prior queues are validator-bound and panels may differ, but an auditor reproduces them exactly from that validator's ordered request anchors and prior bundles.

9.6 Utility and weights

The launch utility is:

```
U_i = max(0, A_i - quality_floor)^2
```

`quality_floor` is the single floor parameter in Section 15 (initially 0.10); the same value gates utility here and eligibility reporting everywhere else. The squared curve increases relative reward differences above the floor.

The floor is calibrated, not assumed. Raw-video ASL translation is a hard task; a floor set above the field's real accuracy distribution would starve the subnet of positive utilities regardless of demand. At publication, no protocol-comparable distribution has yet demonstrated that current models clear `0.10`. UMI translation-weight activation therefore requires published shadow accuracy distributions from SN78 showing at least `2*M_gate` miners sustaining positive utility across the full ten-tempo gate (Section 14). No validator may submit UMI translation weights before that gate and the offline metric-validity gate pass.

If the count of positive utilities falls below the live `MinAllowedWeights` for three consecutive scoring windows on mainnet, that is a declared incident and triggers the scoring-policy revision procedure (a new policy hash with a published activation block). It is never silently absorbed. Validators skip new translation weights during the incident. The Bittensor runtime can still distribute owner alpha through any enabled owner cut, independently of Yuma. It can also distribute validator or staker alpha through its zero-incentive and no-valid-weight fallback rules. Those chain-level transfers are not UMI evidence that translation work cleared the quality floor and MUST be identified separately in public economic telemetry.

Validators submit relative weights proportional to `U_i`. They MUST apply the chain's canonical normalization and quantization. If the number of positive utilities is below the live `MinAllowedWeights`, a validator MUST skip that weight update and emit a health alert. Uniform fallback weights are forbidden.

All score state is keyed by the miner's chain-reported hotkey root at the pinned block. A UID change cannot reset history, and a chain-recorded hotkey successor continues the same root. Histories with different roots MUST NOT merge.

Each validator MUST derive this state from signed source records and its own prior audit bundles. Importing another validator's score database, ranking, or weight vector is a protocol violation.

9.7 Shadow metrics

Validators SHOULD report these separately:

- response latency and timeout rate;
- semantic similarity from a pinned open model;
- BLEU and chrF;
- pose visibility and motion-quality diagnostics;
- per-publisher, per-control-group, per-signer-cohort, and per-stratum scores.

Shadow metrics have zero influence on version 0.1 weights.

9.8 Independent computation, bundle timing, and source monitoring

Deterministic scoring plus full audit disclosure creates a free-riding hazard: a validator could skip challenge work and recompute exact scores from a peer's published bundle. Five controls reduce it:

1. **Chain-timed publication.** A validator MUST NOT publish miner outcomes, scores, or weights before its own `audit_release_block` in Section 10.3. Native timelock concealment therefore remains intact through chain application. A later copy uses stale evidence and remains attributable.
2. **Pre-reveal set anchors.** The finalized assignment-set anchor fixes every initial signed request before issuance. The later request-set anchor fixes its retry transcript before response close. The response-set anchor fixes exactly one sealed envelope or failure marker per request before ground truth opens. A validator cannot invent a better response history after seeing the references.
3. **Copy-proof responses.** A miner's signed answer envelope remains encrypted until the ground-truth reveal, while its ciphertext is anchored after response close. A validator cannot read a non-colluding miner's answer early and insert a later copy.
4. **Self-authenticating work.** Every scored response names the issuing validator inside the miner-signed body (Section 6.3) and answers request bytes signed by that validator's hotkey. A bundle whose responses are bound to another validator's hotkey, or which lacks valid miner signatures over the bundling validator's own requests, is non-conforming on its face. A validator cannot exhibit conforming responses for challenges it never issued. The anchors do not prove network delivery or a validator's local receipt time; those remain explicit residual assertions.
5. **Conditional distinct sampling.** Batch selection is common, while miner ranking is seeded per validator hotkey. When the candidate set exceeds the panel size, conforming validators normally hold overlapping but non-identical response evidence, and a vector copied wholesale from a peer can disagree with the copier's declared panel. When every candidate fits in the panel, membership is identical and this control contributes no detection power.

The chain cannot slash a free rider. The validator-bound signed requests and responses in control 4 remain the primary evidence in both panel regimes: a validator cannot present a peer's responses as its own conforming work. A validator publishing non-conforming evidence fails the conformance requirements of Section 17.

Source-conditioned shadow monitoring. Each validator `v` computes this effect separately for every publisher and every publisher control group. Let `s` denote the source being tested:

```
D_v(i, s, k) = mean_v(i, clips from s in k)
             - mean_v(i, clips from sources outside s in k)

D_v(i, s) = 0.15 * D_v(i, s, fingerspelling)
          + 0.35 * D_v(i, s, short_utterance)
          + 0.50 * D_v(i, s, continuous)
```

The aggregate is reported only when each side of every stratum comparison meets the declared minimum. The validator publishes the component means, sample counts, aggregate effect, and a policy-pinned signer-cluster bootstrap interval. Because requests and histories are validator-bound and panels can differ, each claim is explicitly local to `v` and must be reproducible from that validator's bundle.

The source-monitor bootstrap uses the policy-pinned `umi-source-bootstrap/1` profile. Rows on each side of each stratum are grouped by the bundle's opaque signer-cohort ID. Every replicate independently resamples, with replacement, the same number of signer clusters present on that side and includes every row from a selected cluster. Selecting a cluster more than once repeats all of its rows. For publisher sources, `source_id32` is the publisher account; for control-group sources, it is the control-group ID. The deterministic seed is:

```
source_bootstrap_seed = SHA256(
  "umi-source-bootstrap-seed-v1\0" ||
  window_id ||
  validator_AccountId32 ||
  source_kind_byte ||
  source_id32 ||
  miner_root32 ||
  scoring_policy_hash
)
```

source_kind_byte is 0x00 for a publisher and 0x01 for a control group. For this seed, window_id is the selection window at which the rolling source monitor is evaluated. For zero-based replicate b, side byte z (0x00 for the source and 0x01 for the outside set), zero-based stratum index k in Section 7.2 table order, draw index d, and sorted signer-cluster count m, the selected cluster index is:

```
U64BE_prefix(SHA256(
  "umi-source-bootstrap-draw-v1\0" ||
  source_bootstrap_seed ||
  U32BE(b) || z || U8(k) || U32BE(d)
)) mod m
```

U64BE_prefix interprets the first eight digest bytes as an unsigned big-endian integer, and U8(k) is the single unsigned byte for k. The profile pins the replicate count and confidence level. Each replicate computes all three component effects and their weighted aggregate with exact rationals. Sort the aggregate effects in ascending order. For confidence c and B replicates, the one-sided lower endpoint is the value at one-based rank max(1, ceil((1-c)*B)). Missing cluster IDs, a missing stratum component, or any use of a different resampling rule makes the telemetry non-reproducible and fails the activation gate.

An interval whose lower bound exceeds the alert threshold triggers a public alert and a common follow-up test on future sealed data. It never changes a clip score, A_i, U_i, an issued-assignment denominator, candidate-pool membership, batch validity, or the active publisher registry. Automatic exclusion would require a later protocol amendment with common evidence, a calibrated false-positive bound, and a declared activation block. Version 0.1 therefore exposes publisher-linked anomalies without presenting noisy, validator-specific samples as proof of collusion.

9.9 Offline metric-validity gate

WER and CER become emission-bearing only after they pass a preregistered offline validation. Human judgments establish that the deterministic metric tracks meaning; they never enter live weights or regrade a batch.

The frozen study contains at least 360 previously unseen raw ASL clips from at least 30 signers, with no more than 12 clips per signer. It is signer-disjoint, script-disjoint, and sentence-disjoint from training, model selection, and prior benchmarks. Each stratum contributes 120 clips. A stratum-balanced, signer-level split assigns half to calibration and half to untouched confirmation. Each clip has three through five blind references frozen before candidate outputs. At least six materially different systems, including two independent implementations, translate every clip. Overall results are reweighted to the production 15%/35%/50% stratum mix.

At least six Deaf, ASL-fluent reviewers form the panel, with three independent reviewers per clip and hypothesis. Reviewers see only the raw video and candidate English, with prompt, references, system identity, metric score, and stress-item status hidden. They rate semantic adequacy from 0 through 4 and separately flag errors in actor, action or object, polarity, number, time, and speech act. An output is human-acceptable when at least two reviewers score it 3 or 4 and fewer than two flag a serious error. Ten percent of ratings are blinded repeats. The study is valid only if ordinal Krippendorff's alpha is at least 0.70 overall and 0.60 in each stratum, and binary intra-reviewer agreement is at least 0.80.

The confirmation set also contains blinded meaning-preserving paraphrases for the WER strata, eligible orthographic variants for CER, minimal meaning-changing edits balanced across the declared error types, and fluent but unrelated outputs. Stress labels must be confirmed by the human panel. A metric acceptance threshold is chosen on calibration data and frozen before confirmation opens. On the untouched confirmation split, all of these gates must pass:

- segment-level Spearman correlation between automatic and median human score is at least 0.70, with a signer-clustered 95% lower bound of at least 0.60 overall;
- correlation is at least 0.60, with a lower bound of at least 0.45, in every stratum;
- system-level Kendall tau-b is at least 0.80, with no reversal of a system pair whose human-score difference is significant at 95% under the preregistered signer-cluster bootstrap;
- human-acceptable output recall at the frozen threshold is at least 0.80, with a one-sided 95% lower bound of at least 0.70;
- valid paraphrase and orthographic-variant recall is at least 0.90, with a 95% lower bound of at least 0.85;
- no more than 5% of minimally meaning-changing outputs pass the frozen threshold, with a one-sided 95% upper bound of at most 0.10;

- no fluent unrelated output passes that threshold, with a one-sided 95% upper bound of at most 0.02;
- the automatic score orders valid, minimal-error, and unrelated variants strictly in at least 90% of triplets, with a 95% lower bound of at least 0.85.

A failure keeps UMI translation-weight activation closed. The affected metric may continue as a shadow diagnostic while references, normalization, or scope are revised. Any change requires a new policy hash and a new untouched confirmation set; pass bars cannot be relaxed after results are opened. A narrower launch, such as CER-only fingerspelling, is a new protocol scope rather than an exception to this gate.

10 Chain integration

10.1 Mechanism topology

Version 0.1 uses one mechanism, MechId 0. The owner MUST verify the live mechanism count and UID constraints at a pinned block before UMI weight activation. A validator MUST refuse weight-active mode if the chain topology differs from its configured topology.

10.2 Live state

At the start of each tempo, and again before constructing weights, a validator reads these values from one finalized block:

- runtime spec version, block number, block hash, timestamp, and storage-proof support;
- mechanism count and MechId 0 UID bounds;
- `Tempo`, `LastEpochBlock`, `PendingEpochAt`, `SubnetEpochIndex`, and `BlocksSinceLastStep`;
- weights rate limit, `WeightsVersionKey`, `MinAllowedWeights`, and maximum weight constraints;
- commit-reveal enabled flag, `RevealPeriodEpochs`, and commit-reveal version;
- `Drand.BeaconConfig` and latest stored drand round;
- `Commitments.MaxSpace`, current-epoch `UsedSpaceOf`, `MaxFields`, and the current per-field and total encoding bounds;
- collateral fields and publisher registration ownership;
- subnet active and emission-enabled flags, current owner, `owner_cut_enabled`, live `SubnetOwnerCut`, participant-side alpha emission, root-staker share, Yuma mode and bond parameters, pool reserves and executable quote, and validator take;
- miner root, successor-hotkey, UID, and registration state;
- validator permit, stake, serving requirements, `ActivityCutoffFactorMilli`, the validator's `LastUpdate`, and its existing MechId 0 weight row.

Every logical snapshot MUST identify one block hash, and all state attributed to it MUST come from that block. Mixing values from different heads is non-conforming. The economic fields are telemetry and activation inputs; they never change a miner score or rescue an invalid weight row.

A validator MUST obtain finalized headers through the policy-pinned owned-finality profile, independently of the RPC provider used to retrieve block bodies or storage proofs. The version 0.1 implementation embeds smoldot, connects directly to Finney peers, and accepts no provider RPC URL for finality. Its public evidence is labeled `verifier_attested_finality` and `offline_finality_proof:false`: smoldot verifies warp and GRANDPA data internally, but its public wrapper does not export the portable justification bytes needed for an auditor to verify finality from the bundle alone. The bundle therefore pins and records the reviewed source, lockfile, fixture set, release binary, chain specification, genesis hash, bootstrap header, and hash-chained attestation transcript. It MUST NOT relabel that transcript as a self-contained GRANDPA proof. The transcript and local acceptance receipts are unsigned; their hashes detect alteration relative to a retained run but do not prevent creation of a synthetic offline chain. Activation evidence MUST be captured directly from an invocation of the exact policy-pinned sidecar over its peer-to-peer path. A caller-supplied transcript is replay material only.

Version 0.1 accepts only a governed `grandpa_warp_sync_checkpoint` bootstrap. The policy pins the complete official Finney chain-specification bytes, their subtensor source revision, genesis hash, and the nonzero checkpoint number and hash. The observer independently decodes the checkpoint header and GRANDPA authority set, constructs one deterministic smoldot database, and requires a typed startup receipt showing that this exact checkpoint was selected. Every emitted attestation binds `bootstrap_source:grandpa_checkpoint`, `bootstrap_selected:true`,

and the first verified finalized head. Both that startup head and the first emitted header MUST advance beyond the checkpoint. Genesis fallback, legacy `lightSyncState`, a locally chosen checkpoint, or an ambiguous specification is non-conforming.

The checkpoint and its authority set are an explicit weak-subjectivity assumption and can change only with a new policy hash. They are not proof that an RPC provider reported finality honestly. From the selected checkpoint onward, the owned client verifies peer-supplied warp and GRANDPA data without accepting a provider RPC URL. Provider-supplied storage values become authoritative only after the independent LayoutV1 proof verifier binds their raw `StorageProof` nodes and claimed values to the attested header's state root.

When a verified header enters the validator's durable finality store, the store captures a Unix-millisecond local acceptance time in the same transaction and binds it, the header, the attestation digest, and the preceding receipt into an append-only acceptance digest. Pool-close evidence includes the canonical receipt for the closing header and requires its local acceptance time to precede publication of the selection pulse. This is reproducible evidence of the validator's own event ordering, not a GRANDPA-certified time or a defense against a dishonest validator clock.

Section 10.3 separately defines the current-head schedule snapshot used to build a weight ciphertext; it is not the finalized block used to construct the score row. The scoring policy pins the exact client and timelock-core revisions, content hashes, and epoch-schedule fixtures. Version 0.1 expects commit-reveal version 4, the current reference-client value at publication. The runtime limit of ten unrevealed commits per hotkey and epoch is also pinned by source revision rather than treated as live storage. UMI permits only one. A different live version, failed fixture, or unsupported runtime spec fails closed until a new policy activates.

Startup also fails closed unless at least 300 commitment-space units remain for the assignment, request, and response SHA-256 anchors in Section 8.4, all three calls fit the live field and encoding bounds, the weights rate permits the declared window cadence, and the timelocked queue has room. The effective `activity_cutoff_blocks=max(1, ActivityCutoffFactorMilli*Tempo/1000)` MUST NOT exceed one `window_stride_blocks` interval under the launch policy. The validator exposes the pinned block and every value in health telemetry. Hard-coded epoch position or block timing is forbidden.

Weight-active startup additionally requires both the eligible candidate-miner count and `miner_panel_size` to be at least the live `MinAllowedWeights`. This is a runtime safety floor. Section 14 applies the stricter `2*M_gate` activation test.

10.3 Weight submission

Let `reveal_observation_block` be the first finalized block whose timestamp is no earlier than the window's Quicknet reveal time, and let `weight_commit_ready_height` equal its height plus `weight_commit_buffer_blocks`. The base epoch is `SubnetEpochIndex` at the last finalized block below that height. `weight_commit_open_block` is the first finalized block at or above that height that proves a greater `SubnetEpochIndex`. `weight_commit_epoch_index` is that observed value. This observed transition is authoritative; a predicted tempo boundary is not.

The valid submission interval begins at `weight_commit_open_block` and lasts at most `weight_commit_submission_blocks`. `weight_commit_close_block` is the first finalized block after the open block for which either its height is greater than `openheight+weight_commit_submission_blocks` or its observed epoch index differs from `weight_commit_epoch_index`. A commit is eligible only if its inclusion block precedes that close block and a finalized storage proof shows that the runtime filed it under exactly `weight_commit_epoch_index`. A pulled-forward, deferred, or otherwise shifted transition therefore cannot silently place validators in different epochs. An early, late, or differently keyed submission is non-conforming and cannot be presented as the window's result.

The validator chooses one finalized `weight_build_block` after the ground-truth, spent, and publisher-fault transitions, no earlier than the open block, and before the close block. For every positive-utility miner root, it resolves the root to its chain-recorded successor hotkey and then to that hotkey's MechId 0 UID at this block. It drops an unresolved or ineligible destination, checks `MinAllowedWeights` again, renormalizes, and applies the chain's canonical quantization. The exact root-to-hotkey-to-UID mapping, storage proofs, normalized vector, and quantized row enter the audit bundle.

When the active policy has `translation_weights_active:false`, the validator stops the chain path here. It records the projected row and a finalized event scan proving that its hotkey emitted no UMI weight commit for the window, waits through the derived `weight_commit_close_block`, and publishes a `calibration_no_weight` bundle. It MAY run the pinned ciphertext builder locally for fixture coverage but MUST NOT sign or broadcast the

resulting call. The remaining submission and terminal-state requirements in this section apply only when `translation_weights_active` is true.

Immediately before submission, the validator records one current-head `weight_schedule_snapshot`: its block number and hash, `Tempo`, `LastEpochBlock`, `PendingEpochAt`, `SubnetEpochIndex`, `BlocksSinceLastStep`, `RevealPeriodEpochs`, and actual block time. It supplies those values, the exact UIDs and quantized weights, `WeightsVersionKey`, and validator hotkey public key to the policy-pinned `bittensor_core.get_encrypted_commit_v2`. It then composes the raw mechanism-aware `SubtensorModule.commit_timelocked_mechanism_weights` call with MechId 0, the returned ciphertext and reveal round, and explicit commit-reveal version 4. The current core assumes inclusion at `current_block+1`; later inclusion and terminal checks remain mandatory.

The current convenience `CommitWeights` intent is non-conforming because its preflight and timelock builder independently read best-head state. A future client MAY replace the raw composition only if a policy-pinned fixture proves that it consumes one explicit submission-time schedule snapshot. The weight reveal round is independent of the ground-truth `reveal_round` in `umi-tle/1`; validators MUST NOT substitute one for the other. The snapshot, core inputs, ciphertext, raw call, and test-vector result enter the audit bundle.

Version 0.1 maintains an event-derived commit ledger for each validator hotkey and MechId 0. A hotkey can enter the launch registry only after replay of finalized chain events from CRv4 activation proves that it has never emitted a prior `TimelockedWeightsCommitted` event for this mechanism. The ledger therefore begins empty without assuming that a paged storage-prefix enumeration proves completeness. Before each submission it MUST contain no unresolved entry.

A finalized `TimelockedWeightsCommitted` event adds exactly one entry and changes the submission to `pending`; it does not prove that weights were applied. The validator MUST NOT retry. A finalized proof for the exact `(mechanismstorageindex, weight_commit_epoch_index)` key must contain exactly one matching tuple for its hotkey, ciphertext, reveal round, and commit block. The validator retains another historical proof for that key at the last finalized block of the commit epoch and scans every later finalized commit event for its hotkey and mechanism through removal. Any second event is a protocol failure. Because an accepted entry never moves between epoch keys, its eventual absence is proved at that same key.

The runtime does not store `RevealPeriodEpochs` in the entry. On every reveal block it selects a commit epoch using the live value. The validator therefore records the inclusion-block value, derives an `expected_reveal_epoch` from it, and proves from finalized state and events that the value remains unchanged through entry removal. Any change is a runtime-drift incident. The submission remains nonterminal, and UMI withholds its audit bundle, until a finalized proof shows that the exact entry has left its proven queue. The fixed ciphertext reveal round may already make the weights readable; the runtime cannot retimelock or cancel them. Period drift is therefore also a loss-of-concealment incident and pauses new emission-bearing windows and ordinary weight commits until every affected entry is removed and a compatible policy reactivates.

The validator rechecks its root-to-hotkey-to-UID mapping at the commit-inclusion block. Chain auto-reveal is the expected path, but reveal-time validation can still reject or drop a pending commit. Because the reveal event does not identify a commit hash, the event ledger, exact-key proof, and applied row are part of the terminal classification:

- `applied`: `RevealPeriodEpochs` remained unchanged, the exact entry left its proven queue, a finalized `TimelockedWeightsRevealed` event exists, the stored MechId 0 row exactly matches the expected quantized row, and every destination mapping remains valid at the reveal block;
- `failed`: the exact entry left its proven queue without that event and matching row, a reveal-time validation rejected or dropped it, a destination mapping changed, the applied row differs, the queue was not unique, or `RevealPeriodEpochs` changed before removal.

The runtime's consensus-time masking of UIDs registered or replaced after the commit block is an additional chain defense, not a substitute for these mapping checks. A validator MUST NOT describe a `pending` or `failed` submission as an applied score result.

At `weight_build_block`, the validator classifies its existing row as `previous-row-active` exactly when `LastUpdate+activity_cutoff_blocks` is at least the block height, and records the last UMI bundle that produced that row. Before an ordinary commit, a nonempty row MUST byte-match a named prior `applied` UMI bundle. An unknown or pre-UMI row fails closed and requires a policy-pinned launch recovery; it cannot be adopted as implicit state. This matters because accepting a timelocked commit refreshes `LastUpdate` before the new row is applied. A terminal failure can therefore leave the previous row active. After a failure, the validator pauses ordinary UMI commits until that row is inactive or one policy-controlled recovery submission, built from still-current UMI scores

in a later observed epoch, reaches `applied`. A failed recovery returns to the inactive wait. The incident bundle includes `ActivityCutoffFactorMilli`, the derived cutoff, `LastUpdate`, the previous row, its source window, and active or inactive proofs at every intervening epoch.

For an applied submission, `audit_release_block` is the finalized block containing the verified reveal event, matching row, and entry-removal proof. For a failed submission, it is the first finalized block that proves both the terminal failure and removal of the exact entry from its proven epoch key. If a validator skips before a finalized `TimelockedWeightsCommitted` event, `audit_release_block` is the finalized shared `weight_commit_close_block`. Within one tempo after the applicable block, the validator publishes the audit or incident bundle in Section 12. Before that block it MUST NOT publish miner outcomes, scores, the pre-quantization vector, or the encoded row.

The subnet MUST enable commit-reveal weights before UMI translation-weight activation. Its immunity period MUST exceed `RevealPeriodEpochs*Tempo` so a new miner can receive revealed scores before becoming prunable.

10.4 MEV Shield boundary

MEV Shield provides short-lived mempool confidentiality for eligible transactions. The submission signs the inner extrinsic, encrypts it to the rotating ML-KEM-768 key with XChaCha20-Poly1305, and calls `MevShield.submit_encrypted`. Both the inner and outer mortality eras MUST fit the live runtime maximum, currently eight blocks; the policy-pinned client must pass an expiry fixture. A conforming operator verifies the finalized inner extrinsic and its result; `EncryptedSubmitted` alone is not execution success.

UMI uses this path only for supported coldkey-signed, short-lived operations, including collateral acquisition. As protocol policy, UMI MUST NOT fund or shield a hotkey for weights, commitments, or serving merely to add this wrapper. A future hotkey use requires a policy-pinned capability check, key-separation analysis, and test fixture.

MEV Shield changes transaction transport only. It does not protect off-chain video or references, extend secrecy after block inclusion, guarantee inclusion, replace the future-round ground-truth timelock, or replace native weight commit-reveal.

11 Data policy

11.1 Data classes

Class	Permitted use
Benchmark-only	Challenge delivery, scoring, bounded audit retention
Training-approved	Benchmark use plus model training under the recorded license
Public-release-approved	Training-approved use plus publication under the recorded terms

Permissions are additive only when the consent record explicitly grants them. Benchmark participation alone grants no training or publication right.

11.2 Retention and deletion

Publishers MUST publish retention periods for raw video, derived features, consent records, and audit bundles. A valid deletion request stops future hosting and challenge use where legally and technically possible.

Raw video is delivered to independent miners. The protocol cannot force deletion of copies already received. Consent language MUST state this limitation clearly. Finalized transaction inclusion cannot be deleted. Live `CommitmentOf` storage is overwriteable and may be purged; a published content hash remains verifiable only while its artifact is retained.

11.3 Privacy

Public manifests MUST exclude names, contact details, wallet mappings, private object URLs, and raw consent records. Audit bundles use opaque participant identifiers. Raw video publication requires the public-release permission class.

12 Audit bundle

Within one tempo after its `audit_release_block`, each validator that processed the window, whether it committed or skipped, MUST publish a content-addressed audit or incident bundle. Its manifest contains the protocol and scoring-policy hashes, software revisions, window ID, highest stage reached, terminal classification, `audit_release_block`, and every canonical reason code. For each stage below, the bundle MUST contain its listed evidence if reached. Otherwise it contains an explicit `not_reached` marker naming the prior-stage reason and no fabricated placeholder artifacts. The manifest and all referenced protocol objects, excluding the separately retained raw video objects, MUST fit `maximum_audit_bundle_bytes` without transport compression. Every JSON object is stored as its RFC 8785 UTF-8 bytes and every binary object as the exact protocol bytes. The manifest lists each object's SHA-256, media type, and byte length, sorted by raw digest; duplicate digests are invalid. The accounting value is:

```
audit_bundle_bytes =
  len(UTF8(RFC8785(bundle_manifest))) +
  sum(len(exact_referenced_object_bytes))
```

Exceeding the cap is a validator fault; the assignment preflight in Section 8.4 must prevent it before any request is issued.

1. **Pool and selection:** window index, announcement and closing blocks, derived rounds, policy-declared finality evidence, pool and public batch manifests, closing-block proofs, availability certificate and certified bodies, selection pulse and trace, the exact mirror-discovery rule and canonical video-delivery issuance request, response, and evidence; every bounded successful, failed, or interrupted retrieval and issuance attempt; exactly reconciled observed and ceiling-accounted byte totals; encrypted ground-truth objects; previous and resulting spent roots; and previous and resulting publisher-fault roots with all sorted deltas.
2. **Assignment:** assigned miner roots and closing-block mappings, every exact initial signed request and authentication record, and the assignment leaves and assignment-set root with finalized proofs.
3. **Request transcript:** every retry record, the complete authentication arrays, and the request leaves and request-set root with finalized proofs.
4. **Sealed response:** exactly one canonical `sealed_response_record` and response leaf per request, the response-set root and finalized proofs, every received signed wire envelope, and receipt metadata or declared outer failure.
5. **Reveal and score:** the `umi-tle/1` profile and portable envelopes, verified drand pulse, revealed ground truth and responses with decryption evidence, normalized-text fixtures, per-reference edit distances, canary evidence, per-clip and per-stratum results, rolling-queue state, utility scores, and all source-conditioned shadow telemetry.
6. **Weight build:** every pinned finalized chain snapshot; root-to-hotkey-to-UID mappings and storage proofs; the pre-existing row, activity-cutoff factor and derived blocks, `LastUpdate`, and prior-row classification; final pre-quantization vector; normalization and quantization trace; and expected MechId 0 row.
7. **Commit and terminal state:** the observed commit epoch and open/close proofs; submission-time schedule snapshot; core inputs and output; raw commit call; activation-to-removal event ledger, exact-key inclusion and removal proofs, period-history proof, expected reveal epoch, and finalized `TimelockedWeightsCommitted` event. It then contains the finalized `TimelockedWeightsRevealed` event and matching row, or complete terminal-failure and stale-row evidence.

Another conforming implementation MUST reproduce every clip score, state transition, pre-quantization weight, encoded row, and terminal chain classification for every reached stage exactly. It MUST also reproduce why every later stage was not reached. A mismatch is a validator fault and blocks UMI weight readiness until resolved.

13 Security requirements

Threat	Required control
Label leakage	Opaque IDs, scrubbed metadata, silent video, label-free URLs
Challenge lookup	Fresh human clips, canonical single-use retirement, and permanent publisher-local quarantine after first external disclosure; failed-attempt completeness remains a publisher-trust assumption
Replay or substitution	Video digest, frame digest, derived spent registry, block-bound signatures
Publisher and miner collusion	Control-group caps and mixed-group windows; canaries, shadow alerts, public audit; declared residual risk

Threat	Required control
Publisher reveal sabotage	Objective fault leaves, cooldown after one strike, version-level exclusion after two
Validator cherry-picking	Deterministic batch and miner sampling
Miner impersonation	`btauth/1` plus serving-hotkey response signature
Validator-to-miner answer copying	Miner envelopes sealed until ground-truth reveal; signed ciphertext set anchored after response close
Selective response scoring	Assigned failures included as zero
Post-reveal response fabrication	Finalized request-set and response-set roots before ground truth opens
Prompt injection through output	No LLM judge in the weight path
Synthetic fallback	Production scoring fails closed on missing video, model, reference, or decoder
Missing corpus score	No corpus mechanism and no uniform fallback weights
Weight copying	Fresh rankings plus native timelock concealment through chain application
Validator free-riding on audit bundles	Release after applied reveal; pre-reveal set anchors; validator-bound responses; conditional per-validator sampling when the panel is truncated
Selection grinding	Post-close drand selection round; pool and miner snapshot fixed before the round exists
Pool equivocation	One finalized per-publisher pool anchor and canonical manifest hashing
Local pool omission	Whole-window fail-closed behavior; no validator-local candidate removal
Artifact withholding	Pre-close availability quorum, signer mirrors, and fresh qualification after breach
Validator mirror credential leakage	Disjoint authenticated retrieval and public delivery origins; canonical post-selection issuance; no source URL in miner requests
Timelock profile drift	Pinned envelope fixture and drand tuple checked against activation-block state
Spent-state divergence	Local replay failure skips the update; later peer mismatch pauses subsequent updates
UID reassignment	Root-to-successor-to-UID proofs at build, inclusion, and reveal; mismatch fails the result
Accepted but unapplied weights	Pending, applied, and failed states verified against finalized events and storage
Failed commit refreshes stale row	Prior row, `LastUpdate`, and effective activity cutoff audited; short cutoff; ordinary commits paused after failure
Pre-UMI SN78 weight state	New weight version, event-derived pending-entry cutover audit, and all legacy MechId 0 rows inactive before the first UMI commit
Short-lived mempool inspection	MEV Shield for supported coldkey calls; no claim of post-inclusion secrecy
Data poisoning	Blind review, provenance, signer diversity, control-group caps
Single-use supply exhaustion	Three-batch global cap, at most one selection-driven unused batch in a valid window, independent runway statements, and mainnet-cadence soak
Validator resource exhaustion	Streaming byte ceilings, bounded panel and candidate pool, resource telemetry, and direct-cost coverage gate
Runtime drift	Pinned chain reads, version checks, fail-closed startup

External model access is allowed. Miners remain responsible for data handling, model terms, service availability, and protecting challenge material during the scoring window.

14 UMI weight activation gates

SN78 is registered and chain-active on mainnet. That existing chain state does not by itself activate UMI translation weights. The calibration policy sets `translation_weights_active: false`: validators may execute the challenge, reveal, score, and audit stages on SN78, but MUST NOT submit a UMI weight call. Shadow bundles use the terminal code `calibration_no_weight` and release at the shared `weight_commit_close_block` that would have applied to the window. Any native alpha distributed during this phase is identified as bootstrap emission, not as evidence that UMI translation scoring passed its gates.

At publication, `start_call` has activated SN78, so participant-side alpha epochs continue during calibration. The separate root-controlled `subnet_emission_enabled` flag governs TAO-side pool injection, not participant-side alpha emission. Validators still verify both live states rather than relying on this status line.

A shadow challenge used as activation evidence MUST satisfy every eligibility rule for an emission-bearing challenge; the term names the target protocol class, not a calibration-period payout.

The first conforming UMI weight submission requires a new policy hash with `translation_weights_active: true`, a published activation block, and all of the following. Let `policy_without_activation` be the scoring-policy object after removing exactly its top-level `translation_weights_active` and `activation_block` members:

```
activation_equivalence_digest = SHA256(
  RFC8785(policy_without_activation)
)
```

The shadow policy MUST publish this digest before the 30-day soak. The activated policy MUST reproduce it exactly; only `translation_weights_active` and `activation_block` may differ. Any other policy change, including a registry, cadence, batch, panel, byte, attempt, resource, deadline, scoring, or economics change, invalidates the soak and requires a new one. Live chain values remain subject to the activation-block checks elsewhere in this specification; they are not silently copied into the policy bytes.

The shadow policy pins `soak_start_window_index`. Its announcement block is the soak start. The soak end is the first finalized block whose timestamp is at least `minimum_soak_duration_seconds` after the finalized start-block timestamp. The soak's scheduled-window set contains every consecutive policy window from that index whose derived `weight_commit_close_block` is no later than the soak end; no member may be omitted. A scheduled window is valid only if it produces a nonvoid two-batch scoring cohort and every registered soak validator reaches `calibration_no_weight` with a reproducible bundle. Every skipped, void, unattempted, or capacity-failed member remains in the denominator:

```
valid_window_rate = valid_scheduled_windows / all_scheduled_windows_in_soak
```

Let `M_gate` be the greatest finalized `MinAllowedWeights` value in every logical Section 10.2 chain snapshot required for each scheduled soak window and at the finalized proposed activation block:

```
M_gate = max(MinAllowedWeights_b for every required soak and activation snapshot b)
```

`MinAllowedWeights` remains a live chain input, not an owner-selected scoring-policy constant. A value that makes the miner count, panel size, rolling coverage, or positive-utility gates fail keeps activation closed. It cannot be bypassed by choosing a more favorable snapshot.

Activation drills use one public `umi-activation-drill/1` report per drill. Its logical schema is:

```
{
  "schema": "umi-activation-drill/1",
  "drill_id": "late-response",
  "scoring_policy_hash": "hex-encoded-sha256",
  "activation_equivalence_digest": "hex-encoded-sha256",
  "fixture_manifest_sha256": "hex-encoded-sha256",
  "execution_environment_sha256": "hex-encoded-sha256",
  "expected": {
    "outcome_code": "assignment_zero",
    "reason_codes": ["late"]
  },
  "observed": {
    "outcome_code": "assignment_zero",
    "reason_codes": ["late"]
  },
  "evidence_bundle_sha256": "hex-encoded-sha256",
  "replayers": [
    {
      "administrator": "ss58-independent-replayer",
      "implementation_sha256": "hex-encoded-sha256",
      "result_sha256": "hex-encoded-sha256",
      "scheme": "sr25519",
      "signature": "0x..."
    }
  ]
}
```

The fixture, environment, and expected tuple MUST be published before execution. The environment hash binds the exact runtime or reproducible chain-fork snapshot, mutation driver, policy, and dependency pins. A drill passes only when its observed tuple exactly matches the table below, its referenced audit, incident, event, and storage evidence verifies, and at least two independently administered replayers using independently implemented drivers sign the same `result_sha256`. Replayer implementations derive that value as:

```
result_sha256 = SHA256(
  "umi-activation-drill-result-v1\0" ||
  RFC8785(report_without_replayers)
)
```

For each replayer, let `replayer_attestation` be the RFC 8785 object containing its `administrator`, `implementation_sha256`, `result_sha256`, and `scheme` fields. Its signature follows Section 6.3 over:

```
SHA256(
  "umi-activation-drill-replayer-v1\0" ||
  result_sha256 ||
  RFC8785(replayer_attestation)
)
```

`result_sha256` is decoded to its raw 32 bytes in the signature formula. A missing field, extra or missing reason code, unverifiable evidence object or signature, or replay disagreement fails the drill.

Drill ID	Required pass outcome	Canonical reason codes
`ground-truth-sealing`	Pre-reveal decryption is rejected; the declared round opens exactly the committed plaintext	none
`late-response`	The affected assignment remains in the denominator with score zero	`late`
`early-reveal`	The complete selection window is void	`ground_truth_early_reveal`
`mirror-loss`	Retrieval continues from a certified mirror without changing the candidate pool	none
`certificate-breach`	The affected validator skips the complete window and publishes an incident bundle	`certificate_breach`
`one-validator-loss`	The launch-minimum certificate with the remaining three valid signers still verifies	none
`availability-equivocation`	Conflicting quorum certificates make the window void	`availability_equivocation`
`malformed-pool`	The malformed anchored pool contributes no candidate	`malformed_pool`
`anchor-replacement`	The retained closing-block proof remains authoritative and a later replacement is ignored	none
`assignment-anchor`	The affected validator skips before issuance	`assignment_anchor_invalid`
`request-anchor`	The affected validator skips the window	`request_anchor_invalid`
`response-anchor`	The affected validator skips the window	`response_anchor_invalid`
`response-copy`	A response bound to another validator scores zero for the copied assignment	`response_validator_binding_mismatch`
`spent-replay`	A prior spent hit is ineligible, or a script hit learned after reveal makes the selected window void	`spent_replay`
`publisher-fault-replay`	Independent replay produces the same fault root, strike count, and cooldown state	none
`canary-hit`	The window is void, new active-policy submissions pause, and the hit appears in the incident bundle	`canary_hit`
`publisher-alert`	The exact source-conditioned interval produces public shadow telemetry and no score mutation	`publisher_divergence_alert`
`UID-reassignment`	The pending weight result terminates as failed	`uid_reassignment`

Drill ID	Required pass outcome	Canonical reason codes
`accepted-but-unapplied-weight`	The accepted commit terminates as failed after exact-key removal	`accepted_but_unapplied_weight`
`duplicate-weight-commit`	The duplicate event makes the submission fail and pauses ordinary commits	`duplicate_weight_commit`
`epoch-pull-forward`	The early or differently keyed submission is rejected as the window result	`weight_epoch_pull_forward`
`epoch-deferral`	The late or differently keyed submission is rejected as the window result	`weight_epoch_deferral`
`reveal-period-mutation`	The entry remains tracked through removal, terminates as failed, and new submissions pause	`reveal_period_mutation`
`stale-row`	Ordinary commits remain paused until the prior row is inactive or a controlled recovery applies	`previous_row_still_active`
`legacy-row-cutover`	Activation remains closed while any legacy entry is unresolved or legacy row remains active	`legacy_weight_state_active`
`drand-profile-mismatch`	Startup fails closed before a window is processed	`drand_profile_mismatch`
`MEV-Shield-expiry`	Expired transport is not reported as successful inner-call execution	`mev_shield_expired`
`runtime-upgrade`	An incompatible spec, metadata, interface, or fixture change fails startup pending a new policy	`runtime_upgrade_incompatible`

Published calibration results for batch size, deadlines, quality floor, strata, and miner sampling pass only when they bind the tested candidate policy and show that its chosen values satisfy the metric-validity, positive-utility, vector, valid-window, and resource gates below. An inconclusive report does not pass. The public incident and window-void procedure MUST map every canonical condition to an assignment or window outcome, reason code, publication deadline, submission pause and resume rule, responsible role, and reactivation criterion.

- one mechanism confirmed on the live chain;
- current HTTP authentication and weight interfaces implemented;
- commit-reveal weights enabled, version 4 still live, and the complete epoch- schedule and timelock fixtures verified against one-block snapshots;
- copy-proof miner response envelopes passing cross-implementation encryption, signature, deadline, and decryption fixtures;
- live commitment space and encoding bounds sufficient for all three validator anchors, plus compatible weight-rate and queue capacity;
- observed-epoch, unique-pending-entry, shifted-epoch, and stale-row recovery fixtures passing against the pinned runtime;
- every initial validator hotkey carrying no prior MechId 0 CRv4 commit event and an empty MechId 0 row; later submissions must have zero unresolved ledger entries and satisfy the byte-matched prior-row rule;
- a new UMI `WeightsVersionKey` finalized for SN78, plus an event-derived cutover audit proving that every pre-UMI MechId 0 pending entry is terminal and every pre-UMI MechId 0 row is inactive before the first UMI commit; an active legacy row or unresolved legacy entry keeps translation weights inactive; the audit also publishes storage proofs for the complete activation-block MechId 0 bond state used by the economics replay;
- effective activity cutoff no longer than one window stride;
- immunity period longer than the full configured weight-concealment interval;
- at least four independently administered validators with live permits;
- at least `max(3, 2*M_gate)` active miners from at least two independent implementations, a policy-pinned panel size of at least `2*M_gate`, and enough retained rolling assignments for at least `2*M_gate` miners to satisfy every observation minimum;
- exactly three active challenge-publisher hotkeys across exactly three independently administered control groups at launch, with all common control relationships disclosed, every hotkey meeting the locked-collateral requirement, and the signed runway statements plus realized soak evidence showing that any two groups can sustain the

declared cadence after the third enters cooldown; each future window still requires a fresh availability certificate;
- one valid `umi-validator-capacity/1` statement for every registered soak validator, with the complete set matching the policy's `validator_capacity_set_root`;
- no more than one candidate batch from each active control group and no more than three candidate batches in the complete certified pool for any window;
- ten consecutive shadow tempos on SN78 with exact independent score recomputation and no UMI weight submission;
- a 30-day SN78 shadow supply and validator soak covering at least `minimum_soak_windows` scheduled windows at the proposed activated cadence, publishing aggregate eligible yield, rejection and retirement counts, p50 and p95 response latency, transferred and retained bytes, compute and peak-storage use, audit-bundle size, chain fees, deadline headroom, signed raw meter records, and every Section 6.1 utilization ratio;
- a `valid_window_rate` of at least `minimum_soak_valid_window_rate`, no validator-capacity deadline miss, and the policy-pinned percentile of deadline, compute, memory, transfer, retained-storage, and audit-bundle utilization no greater than `maximum_soak_resource_utilization` of the corresponding policy ceiling or declared validator capacity;
- all validator-vector distance and top-k overlap limits in Section 8.3 passing in every one of those tempos;
- exact agreement on candidate-pool, spent-state, and publisher-fault roots throughout those tempos;
- the WER and CER validity study in Section 9.9 passing every preregistered bar;
- the honest and injected-leak canary studies in Section 7.6 passing for both metrics;
- no synthetic, placeholder, answer-bearing ID, or unrevealed-reference fallback in production code;
- 100% of scored clips carrying eligible consent and provenance records;
- 100% of scored scripts and clips passing the spent-registry check;
- valid public `umi-activation-drill/1` reports for every drill in the table above;
- published calibration results for batch size, deadlines, quality floor, strata, and miner sampling that satisfy the binding and pass criteria above;
- published shadow accuracy distributions meeting the positive-utility requirement of Section 9.6;
- valid `umi-publisher-capacity/1` statements from all three independent groups, covering at least `challenge_supply_runway_days` at the proposed cadence, all delivered and reserved script groups, and the loss of any one group;
- public validator economics reports meeting `validator_cost_coverage` for every validator in the registered soak set under the common conservative case;
- a public incident and window-void procedure containing every required mapping and resume criterion above.

The first ten activated UMI tempos form a mainnet probation window. Every validator update in that period MUST end with a finalized matching `TimelockedWeightsRevealed` event and applied MechId 0 row, and all vector-distance, top-k, candidate-pool, spent-state, and publisher-fault agreement limits above MUST continue to pass. A failure pauses new UMI weight submissions and returns the protocol to a published shadow policy while the incident is resolved.

Bootstrap datasets MAY support model training and shadow evaluation. They are ineligible for activated UMI weight calculation.

15 Pre-weight-activation calibration profile

These values define the version 0.1 SN78 shadow profile. Calibration MAY change them before UMI translation-weight activation through a new scoring policy hash and a published activation block.

Parameter	Initial value
Emission-bearing clips per batch	12
Batches selected per window	2 from distinct publisher control groups
Active publisher hotkeys and control groups (`max_active_publishers`, `max_active_control_groups`)	exactly 3 of each at launch
Candidate batches per publisher and control group per window (`max_candidate_batches_per_publisher`, `max_candidate_batches_per_group`)	at most 1 each
Maximum candidate batches per window (`max_candidate_batches_total`)	3 total

Parameter	Initial value
Maximum unused retirement per valid window	1 batch, 14 clips
Availability quorum and signer mirrors	over two thirds of active validators, minimum 3
Miner panel per validator and selection window	up to 32
Rolling score window (`rolling_batch_count`)	4 valid batches
Score maximum age (`score_max_age_windows`)	4 scheduled windows
Minimum assigned clips	12
Minimum clips per stratum	2
Median pairwise validator-vector TV limit	0.10
Maximum pairwise validator-vector TV limit	0.20
Minimum pairwise top-k overlap	0.80
Quality floor (`quality_floor`, also the utility floor)	0.10
Utility exponent	2
Minimum accepted references	3
Maximum accepted references	5
Maximum reference length (`maximum_reference_utf8_bytes`, `maximum_reference_tokens`, `maximum_reference_graphemes`)	4 KiB UTF-8, 128 normalized tokens, and 512 normalized graphemes excluding whitespace
Maximum hypothesis length (`maximum_hypothesis_utf8_bytes`, `maximum_hypothesis_tokens`, `maximum_hypothesis_graphemes`)	4 KiB UTF-8, 128 normalized tokens, and 512 normalized graphemes excluding whitespace
Maximum signed request body (`maximum_request_body_bytes`)	64 KiB
Maximum signed response envelope body (`maximum_response_body_bytes`)	64 KiB
Maximum HTTP headers per message (`maximum_http_header_bytes`)	16 KiB
Signed-request maximum age and allowed clock skew (`btauth_max_age_seconds`, `btauth_allowed_skew_seconds`)	360 seconds and 2 seconds
Maximum request transmissions per assignment (`maximum_request_transmissions_per_assignment`)	2
Maximum response bodies per assignment (`maximum_response_bodies_per_assignment`)	2
Maximum video fetch attempts per actor, video, and window (`maximum_video_fetch_attempts_per_actor`)	2
Maximum wire bytes per assignment (`maximum_assignment_wire_bytes`)	34 MiB
Maximum wire bytes per validator and window (`maximum_validator_window_wire_bytes`)	40 GiB
Maximum audit bundle excluding raw video (`maximum_audit_bundle_bytes`)	384 MiB
Maximum pool or public batch manifest (`maximum_manifest_bytes`)	256 KiB each
Maximum portable ground-truth envelope (`maximum_ground_truth_envelope_bytes`)	1 MiB
Window stride	360 blocks, matching the initial subnet tempo
Pool proposal close	30 blocks after announcement
Pool anchor close	45 blocks after announcement
Pinned target block interval	12 seconds, verified at activation
Selection finality buffer after expected pool close	300 seconds
Challenge anchor and issue allowance	300 seconds
Response window	300 seconds; 50-block construction-to-deadline bound at the initial interval

Parameter	Initial value
Video delivery grace after response close	60 seconds
Weight-commit buffer after reveal observation	30 blocks
Weight-commit submission interval	30 blocks within one observed epoch
Weight commit-reveal version	4, verified against live state
Weight reveal period	1 subnet epoch, verified against live state
Maximum clip duration	15 seconds
Maximum clip size	16 MiB
Semantic score weight	0
Pose score weight	0
Latency score weight	0
Canary fraction per batch	10%, rounded up; 2 zero-weight canaries at launch
Canary CER hit threshold	0.50, fixed rational pending confirmation
Canary WER hit threshold	0.50, fixed rational pending confirmation
Honest canary comparisons	40,000 per metric with zero hits
Publisher and control-group monitoring window	12 valid batches
Source divergence alert threshold	0.15 lower confidence bound
Divergence minimum sample	6 clips per side and stratum
Source-monitor bootstrap profile	`umi-source-bootstrap/1`; 10,000 signer-cluster replicates per validator, miner root, source, and window
Source-monitor confidence level	0.95 one-sided lower endpoint by the Section 9.8 nearest-rank rule
Publisher minimum locked collateral (`M_alpha`)	set at shadow calibration start
Challenge-supply runway (`challenge_supply_runway_days`)	90 days at the proposed activated cadence
Validator direct-cost coverage (`validator_cost_coverage`)	at least 1.25
Validator economics cost schedule	versioned and hash-pinned before soak
Validator resource-capacity set	one signed statement per registered soak validator; set root pinned before soak
Minimum independent economics price sources	3 per non-chain unit or conversion
Validator executable-quote and TAO-price percentile (`validator_quote_percentile`)	0.10, nearest rank
Conservative validator alpha-value haircut (`validator_alpha_value_haircut`)	0.50 after the minimum activation-block/soak-percentile quote
Activated-parameter supply and validator soak	30 consecutive days
Minimum soak duration (`minimum_soak_duration_seconds`)	2,592,000 seconds
Soak start (`soak_start_window_index`)	fixed in the candidate policy before the soak
Minimum scheduled soak windows (`minimum_soak_windows`)	30
Minimum valid-window rate (`minimum_soak_valid_window_rate`)	0.95
Resource-utilization percentile (`resource_utilization_percentile`)	0.95, nearest rank
Maximum percentile soak resource utilization (`maximum_soak_resource_utilization`)	0.75 of each pinned ceiling or declared capacity
Publisher fault cooldown (`publisher_fault_cooldown_windows`)	4 scheduled windows
Publisher version-level exclusion	second strike
Reveal safety margin after response close	300 seconds

The 360-block stride is an SN78 shadow stress cadence, not an automatically accepted activated challenge-retirement rate. Before the 30-day soak starts, the candidate activation policy MUST choose its tempo and window stride using the available supply and load evidence, then recheck the weight rate, activity cutoff, commit-reveal schedule, score age, and publisher cooldown in blocks and wall-clock time. The soak tests those exact values. Evidence that calls for

any change produces a new shadow policy and restarts the soak. A faster cadence does not become eligible merely because the chain can accept it.

The initial five-minute issue allowance and five-minute response window are calibration candidates, not demonstrated operating margins. They can enter the qualifying soak only when every scheduled window supplies the exact deadline and resource evidence required by Sections 6.1 and 14. A deadline miss, inadequate headroom, or failed utilization gate requires a revised shadow policy and a restarted soak.

16 Extension rules

UMI's long-term objective is a general, verifiable translation layer between human interaction and machine systems. Future mechanisms may interpret human language, motion, gesture, expression, and multimodal context as machine-usable state or intent. Other mechanisms may render machine state, intent, or output as accessible text, speech, signed motion, haptics, or another human-facing form.

Version 0.1 reserves no emission for a future task. An extension must identify a consumer and provide benchmark evidence, a threat analysis, and a migration plan.

Subtensor supports independent mechanisms with separate weights, Yuma runs, bond pools, and emission shares. The live `MaxMechanismCount` is authoritative at an activation block and can be raised through chain governance. The table below is a menu of candidates, not a roadmap. Version 0.1 does not depend on an assumed permanent mechanism limit.

Version 0.1 pins `MechanismCountCurrent` to one at launch and keeps the initial commodity unified. A multi-mechanism expansion MUST account for the live UID budget, governance approval for a cap increase when required, count-change rate limits, split reset behavior, validator cost, and miner specialization before the owner changes the count.

Potential extension mechanisms are:

Extension	Independent value test	Activation blocker
Human-to-machine interaction translation	Human language, motion, gesture, expression, or multimodal context produces structured state or intent used by a measured machine consumer	Consented task-specific ground truth, safety boundaries, and non-circular scoring
Machine-to-human interaction translation	Machine state, intent, or output is rendered in a human-facing form with measured fidelity and comprehension	Target-user evaluation, privacy and safety controls, and reproducible scoring
Model artifact tournament	Public checkpoints improve held-out ASL translation quality	Reproducible packaging, isolated evaluation, and plagiarism controls
Corpus production	New consented data improves a later held-out model or benchmark	Personhood, provenance, deduplication, delayed utility, and collusion controls
Human adjudication	Blind judgments resolve cases deterministic references cannot cover	Reviewer privacy, anchor accuracy, Sybil resistance, and aggregate-only reporting
Organic serving	Signed real demand measures availability and useful throughput	Anti-self-dealing receipts and a ground-truth or quality-audit path

No extension receives a positive split merely because a mechanism slot is available. Its score must remain independently reproducible and its output must have a named consumer.

An emission-bearing semantic metric requires a pinned model and tokenizer, deterministic inference fixtures, adversarial evaluation, and a hard contribution cap. Human judgment requires a separate privacy, personhood, collusion, and reviewer-integrity design. Neither change can alter already committed batches.

17 Conformance summary

A conforming miner:

- serves the versioned authenticated API;

- verifies the video digest;
- timelock-encrypts its bounded English response to the ground-truth reveal and signs the wire envelope;
- returns encrypted explicit failures;
- exposes no answer-bearing fallback.

A conforming validator under a policy with `translation_weights_active:true`:

- reads registration, ownership, collateral, commitment, and metagraph state at one finalized closing block;
- signs at most one availability-set root per window and mirrors every artifact it certifies through reveal;
- reconstructs the canonical candidate pool, spent state, and publisher-fault state;
- selects batches and miners deterministically without local candidate removal;
- obtains canonical short-lived miner delivery URLs after selection without exposing authenticated mirror source URLs;
- anchors its assignment set before issuance, its retry-complete request set before response close, and one sealed response record per request before reveal;
- decrypts miner envelopes and ground truth together only after reveal;
- scores every emission-bearing assignment with the exact public algorithm;
- rejects ineligible, reused, or unverifiable data;
- resolves scored roots to successor hotkeys and MechId 0 UIDs at one finalized weight-build block and verifies those mappings at inclusion and reveal;
- submits exactly one commit in the observed epoch and treats acceptance as pending until its event ledger, exact-key removal proof, finalized reveal event, and the matching applied row agree;
- publishes a reproducible audit or failure bundle only at its `audit_release_block`;
- excludes canaries from every denominator and publishes divergence only as shadow telemetry;
- uses the policy-pinned one-snapshot timelock core and mechanism-aware raw call;
- enforces streaming artifact ceilings and publishes the resource and dividend telemetry required by the operating-economics gate;
- audits the previous row and pauses ordinary commits after terminal failure until the stale row is inactive or a controlled recovery applies.

Under `translation_weights_active:false`, a conforming shadow validator follows the same rules through deterministic weight build, emits no UMI weight commit, and publishes the `calibration_no_weight` bundle defined in Section 10.3.

A conforming publisher:

- supplies eligible consented data;
- fixes references before commitment;
- publishes and mirrors every candidate artifact before pool close;
- obtains the shared quorum availability certificate before anchoring;
- anchors one canonical pool-manifest hash for the selection window;
- declares its complete publisher control group under the active policy;
- exposes the artifacts from which every revealed script group retires;
- records all 14 video hashes, all 14 frame digests, and all 16 actual and reserved script hashes at first external disclosure, permanently excludes them from later proposals, and reports quarantined failed attempts without treating its private ledger as canonical validator state;
- includes the declared canary fraction in every batch;
- proposes at most one launch batch per window and signs the independent capacity statement required for UMI weight activation;
- maintains the required locked collateral and self-lock floor;
- keeps private participant information out of public artifacts.